AXA Seguros México
Documentación Técnica
Sentencias Judiciales 2025

Contenidos

Contenidos 2

Introducción 8

Modelo operativo 9

Problema 9

Solución 9

Beneficios 9

Público objetivo 10

Estándares de usuario 10

Solicitud de acceso 10

Matriz RACI 11

Arquitectura general del sistema 13

SRV-AZOCR-788 14

Lenguaje y tecnologías principales 15

Repositorio de código 16

URL de despliegue 17

SRV-PYT-784 20

Lenguaje y tecnologías principales 20

Repositorio de código 20

URL de despliegue 21

SRV-PYT-783 28

Lenguaje y tecnologías principales 28

Repositorio de código 29

URL de despliegue 30

SRV-PYT-785 39

Internal

Lenguaje y tecnologías principales 39
Repositorio de código 40
URL de despliegue 41
D-JDK-291 44
Lenguaje y tecnologías principales 45
Repositorio de código 45
URL de despliegue 45
Headers 46
Configuración y ejecución local 47
SRV-PYT-788 47
Requisitos 47
Herramientas y librerías utilizadas 48
Pasos para iniciar el servicio 49
SRV-PYT-784 49
Requisitos 49
Herramientas y librerías utilizadas 50
Pasos para iniciar el servicio 51
SRV-PYT-783 52
Requisitos 52
Herramientas y librerías utilizadas 53
Pasos para iniciar el servicio 54
SRV-PYT-785 54
Requisitos 54
Herramientas y librerías utilizadas 55
Pasos para iniciar el servicio 56
D-JDK-291 57
Requisitos 57

Internal

Herramientas y librerías utilizadas 58
Pasos para iniciar el servicio 59
Variables de entorno 60
SRV-PYT-788 60
SRV-PYT-784 61
SRV-PYT-783 61
SRV-PYT-785 64
D-JDK-291 65
Ingesta y procesamiento de documentos 67
Criterios de aceptación de documentos 68
Formatos admitidos 68
Sobre la orientación del documento 69
Sobre el idioma 69
Sobre el nombre del documento 69
Sobre el tamaño 69
Gestión y versión de documentos durante la carga 69
Despersonalización (Capas de censura con Spacy y LLM) 71
Modelos de Spacy 71
Modelo Largo de Lenguaje (LLM y secure-GPT) 72
Proceso de extracción de metadatos 78
Generación de resúmenes con LLM 82
Almacenamiento y BBDD 85
Estructura de almacenamiento 85
Blob 85
Cosmos 86
Azure AI-Search 86
Asistente conversacional 87

Internal

Flujo de consulta 88
Contexto 90
Prompt 91
Uso de historial 91
Guardado de conversaciones 92
Indicador de éxito 94
Seguridad y privacidad 95
Modelo de Spacy 95
Etiquetado de datos 96
Separación de datos 97
Ajuste y evaluación del proceso 97
Resultados 99
Conclusiones 102
Modelo Grande de Lenguage (LLM) 102
Etiquetado de datos 103
Separación de datos 104
Ajuste y evaluación del proceso 104
Resultados 106
Conclusiones 109
Evaluación y métricas 111
Tiempos de procesamiento 111
Embeddings 112
Patron RAG 113
AXA Secure GPT Model Hub 113
Metodología de evaluación 114
Análisis del desempeño de los modelos de embedidos. 116
Resultados 118

Internal

Logs de procesos exitosos y fallidos 119
BI-040-253 Indicadores de éxito para sentencias en curso 121
Sobre el origen de datos 121
Sobre el tablero 122
Objetivo 122
Datos que alimentan el tablero 122
Elementos del tablero 124
Filtros 124
Medidas 125
Visualizaciones 125
Atención de incidencias y Rollback 127
Monitoreo en Openshift 127
Rollback 128
Proceso: Portal AzureDevOps 129
SecureGPT Model Hub 133
Incidencias. 133
¿Cómo proceder en caso de que se retire alguno o algunos de los modelos de Secure GPT Model HUB que usa la aplicación? 133
¿Cuáles son los modelos del Secure GPT Model Hub que actualmente estamos utilizando en el asistente? 134
En caso de que algún modelo del Secure GPT Model Hub que estamos empleando sea retirado, ¿cuáles serían las implicaciones para el funcionamiento del asistente? 135
¿Es necesario realizar alguna acción en particular ante el retiro de un modelo de Secure GPT Model Hub? 135
Incidencia en el servicio con fecha 11 agosto 2025 136
Flujos ETL 138
ETL Tablero 138
ETL Limpieza 140

Internal

Anexo 144
Sentencia antes y después del procesamiento 144
Diagrama de secuencia 146
Prompts de templates utilizados 147
Prompt General 147
Prompt Legal 147
Prompt Médico 147
Troubleshooting 148
403 Forbbiden – Azure AI Search 148
429 NO_QUOTA – Secure GPT 149
Internet Connection Failed 149

Internal

Introducción

El proyecto IA Sentencias Judiciales tiene como finalidad revolucionar el proceso jurídico en AXA Seguros México mediante la automatización de búsquedas y análisis de sentencias judiciales históricas relacionadas con litigios de siniestros, con un alcance de hasta 10 años atrás. Este sistema combina tecnologías de extracción de datos e IA Generativa para proporcionar a los abogados una herramienta de interacción en lenguaje natural, permitiendo acceder rápidamente a información relevante y reducir significativamente los tiempos de búsqueda en casos previamente litigados.

Además, proporciona indicadores estadísticos de éxito que puedan ser utilizados en litigios en curso, así como en la identificación de patrones que ayuden a prevenir futuros conflictos, optimizando la toma de decisiones en defensa y conciliación. La centralización de la información facilita una evaluación rápida y precisa, mejorando la eficiencia del área jurídica y permitiendo una respuesta más ágil ante quejas o litigios.

Este documento presenta no solo el procedimiento que el usuario final experimenta al interactuar con la plataforma —que incluye la carga de documentos, su procesamiento (extracción y despersonalización), almacenamiento de la información extraída y despersonalizada, así como de los metadatos y resúmenes generados para cada documento, la interacción con el asistente conversacional y la visualización de insights en un tablero— sino que también detalla, desde un enfoque técnico, la estructura de cada servicio responsable de llevar a cabo estos pasos.

Internal

Modelo operativo

Problema

El área jurídica de AXA Seguros México enfrenta un desafío crítico en la gestión y análisis de casos de siniestros litigados. Actualmente, el proceso es manual, basado en la revisión de sentencias judiciales similares a los casos en curso. Esta metodología resulta en una búsqueda lenta y laboriosa, que consume recursos significativos y retrasa la toma de decisiones clave. Además, la dificultad para acceder rápidamente a información relevante limita la capacidad de identificar patrones, reducir riesgos y prevenir futuros litigios, lo que impacta directamente en la satisfacción de los asegurados y en los costos asociados a honorarios, intereses y gastos judiciales. La falta de una herramienta eficiente para analizar y extraer insights de la gran cantidad de datos históricos genera una pérdida de oportunidades en conciliaciones, incrementando la exposición de AXA a costos imprevistos y afectando su competitividad en la gestión de litigios.

Solución

El proyecto IA Sentencias Judiciales propone una solución innovadora basada en tecnologías de extracción de datos e IA Generativa para automatizar y optimizar el proceso de búsqueda y análisis de sentencias judiciales históricas. A través de una plataforma inteligente que permite a los usuarios interactuar en lenguaje natural, se facilitará el acceso rápido y preciso a información relevante, filtrando casos similares y proporcionando indicadores de éxito.

Beneficios

La solución antes descrita tiene como beneficios:

- Generar estrategias de defensa frente a inconformidades y tomar decisiones proactivas tendientes a conciliar dichas inconformidades a la brevedad (UNE y CONDUSEF).

Internal

- Consolidar en una herramienta los datos, archivos y sentencias judiciales de siniestros en litigios de AXA Seguros México de manera organizada, con facilidad para su revisión y análisis.
- Identificar oportunamente caminos de acción para una queja de acuerdo a las resoluciones previamente atendidas para resolverlas de la mejor manera posible.

Público objetivo

La herramienta ha sido desarrollada con el propósito de optimizar el proceso manual actualmente realizado por el área de jurídico para el análisis de casos legales relacionados con siniestros en AXA Seguros México. Por lo tanto, su uso está restringido únicamente a **usuarios internos de las áreas de UNE, CONDUSEF, LITIGIOS y CASS**, específicamente a los abogados de AXA responsables de atender dichos casos.

Estándares de usuario

La plataforma está diseñada para interactuar conforme a 3 distintos roles de acceso:

- **Consultor**: Tiene capacidad de interactuar con el asistente, así como de consultar los documentos de referencia.
- **Configurador**: Además de tener acceso al asistente y a la consulta de documentos, este rol también puede cargar nuevos documentos hacia la base de conocimiento del asistente, pero no podrá eliminar ningún contenido.
- **Administrador**: Este rol tiene el mayor nivel de acceso, pues además de realizar consultas al asistente, visualizar los documentos e ingresar nuevos, también puede remover documentos previamente cargados en la base de conocimiento.

Solicitud de acceso

Para solicitar acceso a la plataforma, esta tendrá que buscarse dentro del catálogo de aplicaciones dentro de la plataforma *Silva*. Cada solicitud deberá ir asociada a su respectivo rol de acceso solicitado:

Internal

- **mxapps_djdk291_consulta**
- **mxapps_djdk291_configuración**
- **mxapps_djdk291_administrador**

Matriz RACI

Durante el desarrollo y operación del proyecto es necesaria la definición de roles y responsabilidades, para ello, se considera lo siguiente:

- **Responsable (R):** Son los encargados de desarrollar el trabajo.
- **Aprobador (A):** Son los responsables de aprobar el trabajo realizado.
- **Consultado (C):** Proporciona información/opiniones sobre la tarea.
- **Informado (I):** Se les debe mantener informados sobre el estatus de la tarea.

Debajo se muestran las tareas identificadas, los roles involucrados, y las responsabilidades asignadas a cada rol.

Matriz RACI (*R: Responsible, A: Accountable, C: Consulted, I: Informed*)			
Actividades	**Abogado interno AXA**	**Abogado especialista CONDUSEF**	**Abogado especialista UNE**
Adquisición de información para reportes de siniestros	R	R	A
Procesamiento de información para generar reportes de siniestro	R/A/C	R/C	R/C

Internal

Consulta a los reportes de siniestros generados	R/A/I	R/I	R/I

Las personas identificadas de cada rol son:

- **CTO:** José de Jesús Luna.
- **Líder del proyecto:** Scarlett Téllez.
- **Líder técnico:** Kevin Zazueta
- **Arquitecto de tecnología:** Denis Arias.
- **Arquitecto de seguridad:** Luis Gutiérrez.
- **Equipo de desarrollo:** Gabriela Mundo y Luis Ángel García.
- **Product owner:** Abigail Martínez.
- **Equipo de soporte:** Por definir.

Internal

Arquitectura general del sistema

El proyecto está sustentado por los siguientes desarrollos por orden de aplicación a los documentos:

- Extracción de información por OCR:

  https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-AZOCR-788

- Despersonalización:

  https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-784

- Orquestador de documentos (carga, llamada a servicios de extracción y despersonalización, y almacenamiento):

  https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-783

- Asistente conversacional:

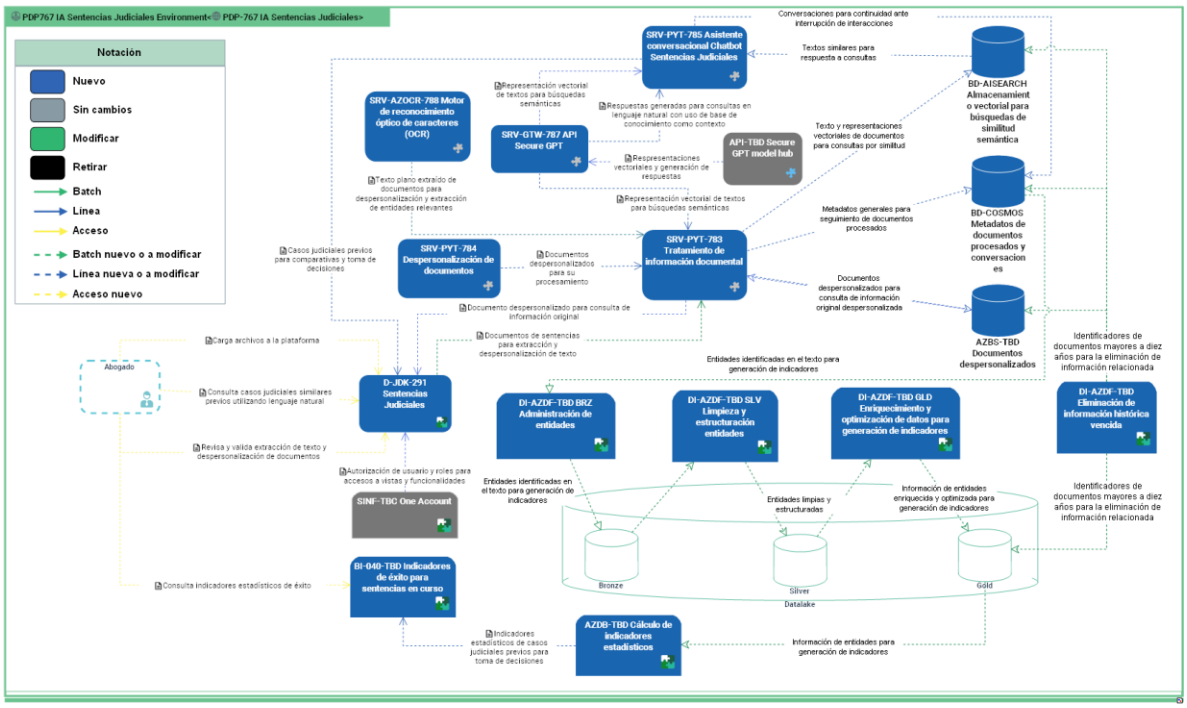
  https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-785

- Interfaz de usuario o front-end:

  https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/D-JDK-291

En el siguiente diagrama de ecosistema se describe la interacción entre los servicios desarrollados y posteriormente se da una explicación detallada de cada uno de ellos:

Internal



SRV-AZOCR-788

El servicio de reconocimiento de caracteres (OCR) se encarga de la extraer la información contenida en documentos PDF, ya sean páginas nativas o escaneadas. Entendiendo por nativas aquellas que han sido generadas digitalmente, en lugar de ser imágenes escaneadas, y que por tanto contienen texto real y seleccionable.

La forma en cómo se clasifican dichas páginas se basa en las siguientes reglas de decisión implementadas que garantizan su adecuado tratamiento y proceso de extracción. Para ello es importante definir las siguientes categorías:

- **Nativo**: PDF original/nativo, es decir, su contenido es texto plano sin alteraciones a la estructura propia de un PDF.
- **No Nativo**: PDF creado a partir del escaneo de un documento físico.

Internal

Las reglas de decisión del sistema están basadas en la presencia de imágenes y texto en la página y su comparación de tamaño relativo, en este sentido, las reglas de decisión son las siguientes:

1. Páginas en blanco, entonces **Nativa**.
2. Si **NO** hay texto y sí hay imágenes, entonces **No Nativa**.
3. Si hay texto y **NO** hay imágenes entonces **Nativa**.
4. Si "no hay imágenes" y el porcentaje de texto del total de la página es menor al 20%, entonces **No Nativa**. Esta regla fue implementada para los casos en que la página está compuesta en su mayoría por tablas.
5. Cuando el porcentaje de las imágenes es mayor al 70% del total de la página, entonces **No Nativa**.
6. Cuando el porcentaje de las imágenes es mayor al porcentaje del texto, entonces **No Nativa**.
7. Cuando el porcentaje del texto es mayor al porcentaje de las imágenes, entonces **Nativa**.
8. Cuando ninguna de las reglas ha entrado en acción, entonces **Nativa**. Esta regla atiende el hecho de evitar aplicar OCR.

Estas reglas son ejecutadas en el orden aquí listado, una vez que una regla entra en acción el resto de ellas ya no se evalúa, de esta forma, se logra su mutua exclusión.

Este servicio es el primer paso para el proyecto PDP-767 IA Sentencias Judiciales ya que al tomar como entrada un PDF se puede identificar, por página del documento, no solo el texto sino también las coordenadas de cada palabra, lo cual será de suma importancia en la despersonalización de la sentencia judicial.

Lenguaje y tecnologías principales

El desarrollo del servicio SRV-AZOCR-788 consiste en una API de extracción de texto implementada en Python y desplegada mediante Docker, utilizando el framework FastAPI.

Internal

El proceso de extracción de texto se basa principalmente en las librerías Fitz y PyMuPDF las cuales se encargan de extraer la información de documentos con extensión **.pdf**, pero no solo el texto en formato seleccionable sino también el contenido en las imágenes presentes en dichos documentos y sus coordenadas correspondientes para poder identificar las posiciones de cada palabra que necesite ser censurada.

Repositorio de código

El desarrollo se encuentra disponible en el siguiente repositorio de código en Azure DevOps:

> SRV-AZOCR-788 Azure DevOps Repo: https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/_git/SRV-AZOCR-788

Dentro de este repositorio se puede identificar la existencia de los siguientes archivos:

- Directorio **"/app/api/":** Contiene todo el código Python desarrollado para el funcionamiento de este servicio.
- Archivo **".env.example":** Sirve como referencia para completar el archivo ".env", pues éste debe contener las variables de entorno necesarias para que la API opere correctamente.
- Archivo **"pyproyect.toml":** Incluye la lista de todas las librerías de Python que deben ser instaladas
- Archivo **"Dockerfile":** Es el archivo de configuración que define la imagen del contenedor con la que la API será contenerizada. Este archivo es usado por el equipo AXA de pipelines para el despliegue de la API en Red Hat OpenShift.
- Archivo **"Docker.dev":** Versión específica del Dockerfile para desarrollo local, que define la imagen del contenedor utilizada para desplegar la API en un entorno de desarrollo de forma local.
- Archivo **"docker-compose.yml":** Configuración para el despliegue local para la API contenerizada.

Internal

URL de despliegue

El desarrollo SRV-AZOCR-788 en su versión QA/PreProd y Prod es desplegado por el equipo AXA de pipelines en Red Hat OpenShift.

Dado que la aplicación fue desarrollada usando FastAPI, es posible revisar sus funcionalidades en las versiones QA y Producción a través Swagger en los siguientes enlaces, respectivamente.

- SRV-AZOCR-788 QA: https://srv-azocr-788-sentenciasjudiciales-preprod-axa-seg-mx.axa-seg-mx-preprod-lpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

- SRV-AZOCR-788 PROD: https://srv-azocr-788-sentenciasjudiciales-prod-axa-seg-mx.axa-seg-mx-prod-hpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

El endpoint que se puede consultar en estas ligas es el siguiente:

- **/ocr/extract/:**

  - Método

    POST

  - Descripción

    Extrae el contenido de un documento PDF cargado y lo devuelve como una respuesta en streaming

  - Input:

    El archivo PDF que se desea procesar y del cual se extraerá el texto.

Internal

- Output:

  La respuesta que contiene los resultados de la extracción en formato JSON. Por cada página se tiene un diccionario con los siguientes atributos:

  - text (str): El texto extraído
  - words (List[Dict]): Lista con un diccionario por cada palabra extraída y sus coordenadas asociadas en el espacio de dicha página
  - metadata (Dict): Los metadatos que incluyen el número de página ("n_page"), las medidas de la misma ("width" y "height"), el número de tokens ("num_tokens"), si la página se identifica como nativa o no nativa ("doc_type"), el ID del documento PDF ("document_id") y unos identificadores por chunks ("n_chunks" y "chunk_id").

    Estos identificadores se basan en 3 parámetros definidos en el archivo *config*.py que se encuentra en el directorio **\app\api**:

    - TEXT_MAX_LENGTH = 1500
    - TEXT_SPLITTER_CHUNK_SIZE = 1000
    - TEXT_SPLITTER_CHUNK_OVERLAP_PERCENT = 0

    Su funcionamiento se basa en lo siguiente:

    1. Si una página tiene menos de 1500 tokens entonces esa página tendrá como identificadores (n_chunks y chunk_id) el valor de 1. Por ejemplo:

Internal

```
{"metadata": {"n_page": 0, "width": 612.0, "height": 792.0,
"num_tokens": 220, "n_chunks": 1, "chunk_id": 1, "doc_type":
"Native", "document_id":
"ad6b2d7915726337678e49a2ffd43b902c4de52fc86ddcb9d6c04354783b1d3b"}
```


2. Si alguna página tiene más de 1500 tokens, tomemos como ejemplo 1700, entonces la información extraída se parte en chunks de 1000 tokens cada uno y por tanto se tendrán 2 pedazos de información o chunks para una misma página, pero seteados de la siguiente forma para ambos chunks, donde claramente cada chunk tendrá su respectivo texto y coordenadas:


```
{"metadata": {"n_page": 0, "width": 612.0, "height": 792.0,
"num_tokens": 1000, "n_chunks": 2, "chunk_id": 1, "doc_type":
"Native", "document_id":
"ad6b2d7915726337678e49a2ffd43b902c4de52fc86ddcb9d6c04354783b1d3b"}
```

```
{"metadata": {"n_page": 0, "width": 612.0, "height": 792.0,
"num_tokens": 700, "n_chunks": 2, "chunk_id": 2, "doc_type":
"Native", "document_id":
"ad6b2d7915726337678e49a2ffd43b902c4de52fc86ddcb9d6c04354783b1d3b"}
```


El procesamiento de otras extensiones, como docx, se hace desde el servicio de documentos (SRV-PYT-783). En él se lleva a cabo una conversión a PDF para después poder llamar a este endpoint y poder extraer la información solo para esta extensión.

- Modelo de datos de solicitud
  file (UploadFile)
- Modelo de datos de respuesta
  StreamingResponse

Internal

SRV-PYT-784

El servicio de despersonalización de documentos busca ser una solución transversal para identificar información sensible dentro de texto plano y censurarla, y si se cuenta con las coordenadas de las palabras dentro de un PDF, generar un nuevo PDF con la información sensible censurada.

El objetivo principal es hacer que ninguna persona pueda ser identificada.

Lenguaje y tecnologías principales

El desarrollo del servicio SRV-PYT-784 consiste en una API de despersonalización de información implementada en Python y desplegada mediante Docker, utilizando el framework FastAPI.

El proceso de despersonalización de texto se basa principalmente en los modelos de Spacy. Esta librería de Python diseñada para ser utilizada en productos reales, provee una variedad de modelos estadísticos y pipelines pre-entrenados en diferentes idiomas y para distintas tareas de Procesamiento de Lenguaje Natural (NLP).

Esta librería fue seleccionada para el proyecto debido a su alta velocidad y a la calidad de sus modelos de redes neuronales, los cuales permiten realizar tareas como el etiquetado, análisis sintáctico, reconocimiento de entidades nombradas, clasificación de textos y otras funciones avanzadas. En este sentido, ayuda a identificar las siguientes entidades dentro de la sentencia judicial: [PERSON] para nombre propios, [ORGANIZATION] para empresas y organizaciones, [ID] para identificadores como el RFC, [EMAIL] para correros y [POLICY] referenciando números de pólizas o contratos.

Repositorio de código

El desarrollo se encuentra disponible en el siguiente repositorio de código en Azure DevOps:

Internal

SRV-PYT-784 Azure DevOps Repo: https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-784/_git/SRV-PYT-784

Dentro de este repositorio se puede identificar la existencia de los siguientes archivos:

- Directorio **"/app/api/":** Contiene todo el código Python desarrollado para el funcionamiento de este servicio.
- Archivo **".env.example":** Sirve como referencia para completar el archivo ".env", pues éste debe contener las variables de entorno necesarias para que la API opere correctamente.
- Archivo **"pyproyect.toml":** Incluye la lista de todas las librerías de Python que deben ser instaladas
- Archivo **"Dockerfile":** Es el archivo de configuración que define la imagen del contenedor con la que la API será contenerizada. Este archivo es usado por el equipo AXA de pipelines para el despliegue de la API en Red Hat OpenShift.
- Archivo **"Docker.dev":** Versión específica del Dockerfile para desarrollo local, que define la imagen del contenedor utilizada para desplegar la API en un entorno de desarrollo de forma local.
- Archivo **"docker-compose.yml":** Configuración para el despliegue local para la API contenerizada.

URL de despliegue

El desarrollo SRV-PYT-784 en su versión QA/PreProd y Prod es desplegado por el equipo AXA de pipelines en Red Hat OpenShift.

Dado que la aplicación fue desarrollada usando FastAPI, es posible revisar sus funcionalidades en las versiones QA y Producción a través Swagger en los siguientes enlaces, respectivamente.

Internal

- SRV-PYT-784 QA: https://srv-pyt-784-sentenciasjudiciales-preprod-axa-seg-mx.axa-seg-mx-preprod-lpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

- SRV-PYT-784 PROD: https://srv-pyt-784-sentenciasjudiciales-prod-axa-seg-mx.axa-seg-mx-prod-hpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

Mediante estos enlaces se puede interactuar con la aplicación y sus endpoints, que se explican a continuación:

- **/censor/label/:**

  - Método

    POST

  - Descripción

    Endpoint para etiquetar las palabras del texto y, en función de dichas etiquetas, generar una versión censurada del texto original.

  - Input

    Diccionario con las siguientes claves:

    - texts (List[str]): Lista de textos a procesar
    - censurable_entities (List[EntityInfo]): Lista de entidades a censurar dados los atributos específicos *type* o tipo de entidad, *replace_with* o la etiqueta con la que será reemplazada dicha palabra, y *collapse* siendo un booleano que indica si la etiqueta se colocará por frases completas. Por ejemplo, si collapse es true y type es “PER”, para el texto “Fernando Gómez es una tienda de ropa” y replace_with como [PERSON], el texto

Internal



censurado obtenido será “[PERSON] es una tienda de ropa”. En cambio, si collapse es false, entonces el texto censurado generado será “[PERSON] [PERSON] es una tienda de ropa”, coincidiendo el número de etiquetas con las palabras censuradas.

- custom_detections (List[CustomDetectionInfo]): Lista de detecciones personalizadas.
- words_white_list (List[str]): Lista de palabras permitidas, en caso de que haya alguna que nunca necesite ser censurada.
- collapsable_words (List[str]): Lista de palabras que pueden colapsarse.
- max_collapse_iters (int): Número máximo de iteraciones para el colapso.
- keep_format (bool): Indica si se quiere mantener el formato original del texto.

Ejemplo:

```
{"texts": ["Fernando Gómez es una tienda de ropa"], "censurable_entities": [{"type": "PER", "replace_with": "[PERSON]", "collapse": false} ], "custom_detections": [], "words_white_list": [], "collapsable_words": [], "max_collapse_iters": 10, "keep_format": false }
```

- Output

Stream de objetos JSON que contiene el texto original, todas las entidades que el modelo de spacy identificó, y el texto censurado.

Hay que notar que además de las etiquetas que nosotros definimos ([PERSON], [ORGANIZATION], [ID], [EMAIL] y [POLICY]), el modelo al ser de reconocimiento de entidades nombradas, también identifica otros tipos como: Entidades Geopolíticas (GPE), es decir países y

Internal

ciudades, Lugares (LOC) que no son GPE, Productos (PRODUCT), Eventos (EVENT), Obras de arte (WORK_OF_ART), y entidades de tiempo y cantidad como Fechas (DATE), Tiempo (TIME), Dinero (MONEY), Porcentajes (PERCENT), y Cantidades (QUANTITY), por lo que **para cada entidad reconocida** por dicho modelo se obtiene un diccionario con la palabra reconocida, el tipo de entidad reconocida, si debe censurarse, y las posiciones que esas palabras toman en el texto original.

Por ejemplo, para un texto dado, en la variable “entities” se obtiene un diccionario para cada entidad reconocida:


```
{"text": "Fernando Gómez es una tienda de ropa", "custom_entities": [], "entities": [{"text": "Fernando Gómez", "type": "PER", "censoring": true, "custom": false, "words": [0, 1]}], "censored_text": "[PERSON] [PERSON] es una tienda de ropa" }
```


- Modelo de datos de solicitud

  LabelRequest

- Modelo de datos de respuesta

  StreamingResponse

- **/censor/document/:**

  - Método

    POST

  - Descripción

    Con base en un documento PDF y una lista de entidades a censurar, genera la versión despersonalizada de dicho documento

  - Input

Internal



- El documento PDF a despersonalizar
- Un JSON que por cada página contiene:
    - metadata (Dict[str, Any]): Los metadatos
    - text (str): El texto extraído
    - words (List[WordDescriptor]): Las coordenadas de cada palabra
    - entities (List[MarkedEntityDescriptor]): Las entidades reconocidas
    - censored_text (str): El texto censurado:

- Output

  El documento PDF que recibe de entrada pero en su versión despersonalizada

- Modelo de datos de solicitud

  file (UploadFile)

- Modelo de datos de respuesta

  CensoreExtraction

Dentro del código, y aunque no es visible en los endpoints porque Swagger no permite esta interacción. Se desarrolló la versión de este endpoint mediante WebSockets.

Los **WebSockets** son un protocolo que permite establecer una conexión **persistente y bidireccional** (full-duplex) entre un cliente (como un navegador web) y un servidor, facilitando la **comunicación en tiempo real** mediante el intercambio continuo de datos. Lo hacen estableciendo primero una conexión mediante un enlace HTTP y luego manteniendo esa conexión abierta, lo que elimina la necesidad de solicitudes y respuestas repetidas, a diferencia del protocolo HTTP tradicional. Esta tecnología es crucial para aplicaciones como chats, juegos en línea, plataformas de trading y

Internal

herramientas de colaboración, ya que permite actualizaciones instantáneas y una interacción fluida entre el cliente y el servidor.

Entonces, la funcionalidad de este endpoint con WebSockets, que se encuentra en el script document.py dentro del directorio **\app\api\v1\routers**, es análoga. Las principales diferencias radican en que, una vez aceptada la conexión e iniciada la comunicación bidireccional, se reciben los bytes de un documento PDF y, durante el proceso, se envían mensajes de estado como FILE_RECEIVED, EXTRACTION_INFO_RECEIVED, INFORMATION_COMPLETE y CENSORED_DOCUMENT. Esto es importante porque la función no devuelve un valor explícito, pero interactúa con el cliente enviando y recibiendo estos mensajes a través del WebSocket, lo que permite monitorear el avance del proceso. Además, envía los bytes del documento censurado y cierra la conexión al finalizar.

- **/censor/text/:**

  - Método

    POST

  - Descripción

    Genera la versión despersonalizada del texto plano que reciba como input, basándose en la variable “extracted_entities” que serán las entidades a censurar.

  - Input

    Diccionario con las siguientes claves:

    - texts List[str]: Lista de textos a procesar
    - censurable_entities List[EntityInfo]: Lista de entidades a censurar dados los atributos específicos *type* o tipo de entidad, *replace_with* o la etiqueta con la que será reemplazada dicha palabra, y *collapse* siendo un booleano que indica si la etiqueta se colocará por frases completas. Por ejemplo, si collapse es

Internal



true y type es “PER”, para el texto “Fernando Gómez es una tienda de ropa” y replace_with como [PERSON], el texto censurado obtenido será “[PERSON] es una tienda de ropa”. En cambio, si collapse es false, entonces el texto censurado generado será “[PERSON] [PERSON] es una tienda de ropa”, coincidiendo el número de etiquetas con las palabras censuradas.

- extracted_entites List[List[ExtractedEntity]]: Lista con una lista por página con las entidades que se censuran, es decir, como el atributo “entities” del output obtenido por **/censor/label/**
- collapsable_words (List[str]): Lista de palabras que pueden colapsarse.
- max_collapse_iters (int): Número máximo de iteraciones para el colapso
- keep_format (bool): Indica si se quiere mantener el formato original del texto.

- Output

  Diccionario con el texto despersonalizado, por página del documento.

- Modelo de datos de solicitud

  CensorTextRequest

- Modelo de datos de respuesta

  StreamingResponse

Internal

SRV-PYT-783

El servicio de administración de documentos es una funcionalidad exclusiva del proyecto PDP-767 IA Sentencias Judiciales. Su propósito es gestionar toda la información relacionada con los documentos utilizados para este proyecto, es decir:

- Orquesta las funciones para la extracción de texto de los documentos (SRV-AZOCR-788)
- Orquesta los servicios para el censurado de información (SRV-PYT-784)
- Administra la información en la base de conocimiento del asistente
- Provee la información almacenada en la base de conocimiento

Por lo que, para asegurar el funcionamiento de este servicio es importante asegurar que los componentes de despersonalización de documentos y el motor de reconocimiento óptico de caracteres también estén en funcionamiento.

Más adelante, en la sección **Ingesta y procesamiento de documentos**, se enlista el proceso completo que se sigue para cada sentencia judicial y en el cual este servicio tiene presencia. Pero, a grandes rasgos al orquestar funciones de los servicios SRV-AZOCR-788 y SRV-PYT-784, se encarga de tomar las salidas de estos servicios y almacenarlas en las bases de datos correspondientes. Además, realiza llamadas a secureGPT para aplicar una segunda capa de despersonalización, obtener las entidades relevantes o metadatos del documento y los resúmenes que ayudan al asistente para tener un contexto de cada sentencia.

Lenguaje y tecnologías principales

El desarrollo del servicio SRV-PYT-783 consiste en una API que orquesta las funcionalidades desarrolladas en los dos servicios anteriormente descritos, es también desarrollada en Python y desplegada mediante Docker, utilizando el framework FastAPI.

Internal

Dado que este proyecto, y este servicio, son potenciados por inteligencia artificial, es necesario contar con acceso a Secure GPT model hub.

Repositorio de código

El desarrollo se encuentra disponible en el siguiente repositorio de código en Azure DevOps:

SRV-PYT-783 Azure DevOps Repo: https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-783/_git/SRV-PYT-783

Dentro de este repositorio se puede identificar la existencia de los siguientes archivos:

- Directorio **"/app/api/":** Contiene todo el código Python desarrollado para el funcionamiento de este servicio.
- Archivo **".env.example":** Sirve como referencia para completar el archivo ".env", pues éste debe contener las variables de entorno necesarias para que la API opere correctamente.
- Archivo **"pyproyect.toml":** Incluye la lista de todas las librerías de Python que deben ser instaladas
- Archivo **"Dockerfile":** Es el archivo de configuración que define la imagen del contenedor con la que la API será contenerizada. Este archivo es usado por el equipo AXA de pipelines para el despliegue de la API en Red Hat OpenShift.
- Archivo **"Docker.dev":** Versión específica del Dockerfile para desarrollo local, que define la imagen del contenedor utilizada para desplegar la API en un entorno de desarrollo de forma local.
- Archivo **"docker-compose.yml":** Configuración para el despliegue local para la API contenerizada.

Internal

URL de despliegue

El desarrollo SRV-PYT-783 en su versión QA/PreProd y Prod es desplegado por el equipo AXA de pipelines en Red Hat OpenShift.

Dado que la aplicación fue desarrollada usando FastAPI, si se requiere revisar funcionalidades, esto es posible a partir de los siguientes enlaces en sus versiones QA y Prod, respectivamente.

- Swagger SRV-PYT-783 QA: https://srv-pyt-783-sentenciasjudiciales-preprod-axa-seg-mx.axa-seg-mx-preprod-lpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

- Swagger SRV-PYT-783 PROD: https://srv-pyt-783-sentenciasjudiciales-prod-axa-seg-mx.axa-seg-mx-prod-hpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

Mediante estos enlaces se puede interactuar con la aplicación y sus endpoints, que se explican a continuación, a través de Swagger:

- **/document/convert-to-pdf/:**
  - Método

    POST
  - Descripción

    Convierte un archivo cargado con extensión distinta a PDF a bytes de PDF y permite descargar dicha versión del documento convertido.

    Solo se admiten extensiones .docx, jpg, png y jpeg.

Internal

- Input

  Documento para convertir

- Output

  La respuesta que contiene los bytes del PDF. En Swagger es posible descargarlos como un documento.

- Modelo de datos de solicitud

  file (UploadFile)

- Modelo de datos de respuesta

  Response

- **/document/extract/edit/:**

  - Método

    POST

  - Descripción

    Extrae el *texto* **sin censurar** de un archivo con extensión .pdf, docx, jpg, png o jpeg utilizando el servicio OCR (SRV-PYT-788). Es decir, mediante la funcionalidad proporcionada en **/document/convert-to-pdf/** se convierte el documento a PDF y posteriormente se procesa con **/ocr/extract/** para obtener el texto y los metadatos de cualquier archivo con las extensiones mencionadas.

  - Input

    Sentencia judicial de la cual se requiere realizar la extracción del texto

  - Output

    Una respuesta en streaming con los datos extraídos por página en formato JSON. Estos datos son los siguientes:

Internal

- metadata (Dict): la que arroja el endpoint **/ocr/extract/**
- text (str): el texto plano extraído
- words (List[Dict]): una lista de diccionarios, donde cada diccionario corresponde a una palabra e incluye tanto la palabra en sí como sus coordenadas espaciales dentro de la página.

- Modelo de datos de solicitud

  file (UploadFile)

- Modelo de datos de respuesta

  StreamingResponse

- **/document/extract/:**
  - Método

    POST

  - Descripción

    Extrae el texto de un documento utilizando el servicio de OCR (SRV-PYT-788) y luego lo procesa a través del servicio de despersonalización (SRV-PYT-784).

    El proceso es similar al endpoint **/document/extract/edit/**, pero ahora también se **despersonaliza** la información y por tanto en la salida se obtiene además "censored_text" que es el texto despersonalizado (ya pasado por las 2 capas), y "entities" que son las entidades identificadas.

    También extrae las entidades relevantes.

  - Input

    Sentencia judicial de la cual se requiere realizar la extracción del texto

Internal

- Output

  Una respuesta en streaming con los datos extraídos por página en formato JSON. Estos datos son los siguientes:

  - metadata (Dict): la que arroja el endpoint **/ocr/extract/**
  - text (str): el texto plano extraído
  - words (List[Dict]): una lista de diccionarios, donde cada diccionario corresponde a una palabra e incluye tanto la palabra en sí como sus coordenadas espaciales dentro de la página.
  - entities (List[Dict]): Lista de diccionarios, en donde cada diccionario corresponde a una entidad reconocida, el tipo de entidad que le corresponde, si debe censurarse, y las posiciones que esas entidades toman en el texto original.
  - censored_text (str): la versión despersonalizada del texto plano extraído

  Además, devuelve un diccionario con las entidades relevantes encontradas por el Large Language Model (llm), en este caso secureGPT: magistrado ponente, circuito o localidad, tribunal (número y materia), fecha de sentencia y la resolución de la misma, la cual toma el valor **1 cuando Axa Seguros México ha perdido el caso**

- Modelo de datos de solicitud

  file (UploadFile)

- Modelo de datos de respuesta

  StreamingResponse

Internal

- **/document/save/:**
    - Método

        POST

    - Descripción

        Se encarga de guardar toda la información extraída y por generar con los modelos de lenguaje. Más específicamente el proceso que sigue es el siguiente:

        1. Obtiene el hash del documento que se ha cargado, el cual funge como su ID, y hace una búsqueda de dicho valor en la base de datos para validar que no haya sido ya procesado el documento.
        2. Guarda en Cosmos DB las entidades relevantes que se obtienen con secureGPT usando el endpoint **/document/extract/**
        3. Guarda el documento en su versión despersonalizada generado con el endpoint **/censor/document/** en Blob Storage
        4. Toma el texto censurado que ya ha sido pasado por las dos capas y genera un resumen de la sentencia, también usando la API de secureGPT, y lo guarda en Blob Storage
        5. Guarda los vectores del texto en AI-Search
        6. Almacena en una tabla de Blob Storage los metadatos en el formato que serán consumidos por PowerBI

    - Input
        - user_id (str): ID del usuario
        - automatic_validation (bool): Indica si el documento cargado se hizo mediante validación automática o manual
        - file (UploadFile): La sentencia judicial que se procesará
        - extraction_json (CensoreExtraction): La salida del texto extraído y metadatos obtenidos por el enpoint **/document/extract/**

Internal



- relevant_entities (Dict): Las entidades relevantes de la sentencia que también se obtienen del endpoint **/document/extract/**

- Output

  Muestra un mensaje de según la información se haya guardado con éxito o no.

- Modelo de datos de respuesta

  StreamingResponse

Este endpoint también cuenta con su versión con WebSockets, que en realidad es la versión utilizada en el proyecto, para optimizar los tiempos de comunicación. Es importante mencionar que la lógica es la misma que la implementada con el método POST ya descrito.

- **/document/{document_id}/:**

  - Método

    GET

  - Descripción

    Permite traer de la base de conocimiento de Blob Storage la sentencia judicial en su versión censurada a través de una búsqueda por su ID (o HASH).

    Este endpoint es usado para la visualización del documento en la plataforma.

  - Input

    - document_id (str): El identificador del documento que se necesita consultar

  - Output

    En Swagger se podrá descargar el documento

Internal

- Modelo de datos de respuesta

  Response

- **/document/page/{page_number}/records/{num_records}:**
  - Método

    GET

  - Descripción

    Enlista los metadatos guardados en Cosmos DB. Toda la información referenciada a algún documento en Cosmos también se encuentra en la base vectorial de AI-Search y en Blob Storage, si se usó el endpoint **/document/save/**

    Su implementación es más visible en el paginado de la base de conocimiento que tiene el asistente, pues mediante este endpoint es posible mostrar en una tabla de 5, 10 o 15 registros, los documentos guardados.

  - Input
    - page_number (int): El número de página del paginado de la base de conocimiento
    - num_records (int): La cantidad de documentos mostrada por paginado
  - Output

    La lista con los IDs de los documentos a mostrar, su mime type y su tamaño.

  - Modelo de datos de respuesta

    Response

Internal

- **/document/records/count:**
  - Método

    GET
  - Descripción

    Devuelve el número de documentos guardados en Cosmos DB
  - Output

    Un int que indica el número de documentos que ya se encuentran en Cosmos DB
  - Modelo de datos de respuesta

    Response

- **/document/{file_id}:**
  - Método

    DELETE
  - Descripción

    Dado el ID de un documento, elimina su versión despersonalizada de Blob Storage, sus embeddings asociados de AI-Search, y sus metadatos (o entidades relevantes) y resúmenes de Cosmos DB. También elimina el registro asociado a la tabla que almacena los metadatos que serán consumidos por PowerBI.
  - Input
    - file_id (str): El identificador único del documento a eliminar de la base de conocimiento
  - Output

    Un mensaje que indica si la información fue eliminada con éxito o hubo alguna falla en el proceso de eliminación.

Internal

- Modelo de datos de respuesta

  Response

- **document/clean-up-data/execute**
  - Método

    DELETE

  - Descripción

    Realiza un proceso de limpieza de documentos en la aplicación que ya no son necesarios por su antigüedad. El proceso consiste en recuperar todos los documentos con sus identificadores y fechas, evaluar si deben ser eliminados según reglas definidas, y proceder a eliminarlos en caso de que corresponda. La función también maneja excepciones y registra los pasos y errores del proceso.

  - No requiere input
  - Output

    Un mensaje de confirmación si el proceso se completa correctamente. En caso de error, lanza una excepción HTTP con detalles del error.

  - Modelo de datos de respuesta

    Response

- **/document/create-table:**
  - Método

    POST

Internal

- Descripción

  Crea una tabla inicial a partir de los metadatos cargados en CosmosDB. Esta tabla ya cuenta con las transformaciones necesarias para que solo sea consumida por PowerBI. En caso de que no haya metadatos almacenados en CosmosDB, entonces genera una tabla vacía, pero con los campos listos para comenzar a llenarse con la carga de documentos.

- No requiere input
- Output

  Un mensaje que indica si la tabla fue generada con éxito o hubo alguna falla en el proceso de generación.

- Modelo de datos de respuesta

  Response

SRV-PYT-785

En este servicio, el usuario puede interactuar con un Modelo Largo de Lenguaje (LLM) capaz de consultar extractos relevantes de texto relacionados con el tema de interés, accediendo a la información previamente almacenada en una base de conocimientos para ofrecer respuestas basadas en esta. Para el proyecto PDP-767, la base de conocimientos está conformada por casos legales que la aseguradora atendió en el pasado y que se encuentran en forma de sentencias judiciales.

Lenguaje y tecnologías principales

El desarrollo SRV-PYT-785 consiste en API conversacional desarrollada en Python y desplegada mediante Docker, utilizando el framework FastAPI. Esta API se fundamenta

Internal

en el patrón RAG (Retrieval-Augmented Generation), que combina la generación de texto con la recuperación de información.

Repositorio de código

El desarrollo SRV-PYT-785 cuenta con un repositorio de código en Azure DevOps, para acceder a dicho repositorio se visita el siguiente enlace:

- SRV-PYT-785 Azure DevOps Repo: https://dev.azure.com/AXA-MX-TECNOL-INFORMACION/SRV-PYT-785

Al revisar el repositorio, podremos encontrar información relevante, de esta información se destacan el siguiente directorio y archivos:

- Directorio **"/app/api/":** El directorio que contiene todo el código desarrollado para que la API sea funcional.
- Archivo **"pyproyect.toml":** La lista de todas las librerías de Python que deben ser instaladas
- Archivo **".env.example":** Archivo de ejemplo que muestra las variables de entorno necesarias para que la API sea funcional.
- Archivo **"Dockerfile":** Archivo Dockerfile que define la imagen de contenedor con la que la API será contenerizada. Este archivo es usado por el equipo AXA de pipelines para el despliegue de la API en Red Hat OpenShift.
- Archivo **"Docker.dev":** Archivo Dockerfile en su versión específica para desarrollo local, este define la imagen de contenedor con la que la API será contenerizada. Este archivo es usado para desplegar la API de forma local.
- Archivo **"docker-compose.yml":** Configuración de despliegue local para la API contenerizada.

URL de despliegue

El desarrollo SRV-PYT-785 en su versión QA/PreProd y Prod es desplegado por el equipo AXA de pipelines en Red Hat OpenShift, debido a su construcción en FastAPI, dicho desarrollo cuenta con una interfaz Swagger que permite la interacción con la API.

Para acceder a la interfaz Swagger de la API se debe consultar alguno de los siguientes enlaces en función de la versión (QA o Prod) con la que se desee interactuar:

- Swagger SRV-PYT-785 QA: https://srv-pyt-785-sentenciasjudiciales-preprod-axa-seg-mx.axa-seg-mx-preprod-lpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs
- Swagger SRV-PYT-785 PROD: https://srv-pyt-785-sentenciasjudiciales-prod-axa-seg-mx.axa-seg-mx-prod-hpl-int.gold.eastus2.azure.openpaas.axa-cloud.com

A continuación, se muestran las rutas definidas en la API y una descripción de sus funcionalidades:

- **/conversation/:**
  - Método

    POST
  - Descripción

    Inicia una conversación.
  - Modelo de datos de solicitud

    InitiateConversationRequest.
  - Modelo de datos de respuesta

    InitiateConversationSessionResponse.

Internal

- **/conversation/{conversation_session_id}/message:**
    - Método

        POST

    - Descripción

        Envía un mensaje al patrón RAG. Este es el core de la aplicación, al consumirlo estamos interactuando con los modelos de IA.

    - Modelo de datos de solicitud

        ConversationRequest.

    - Modelo de datos de respuesta

        Respuesta en streamming.

- **/conversation/{conversation_session_id}/save:**
    - Método:

        POST

    - Descripción

        Persiste la conversación en Azure Cosmos DB.

    - Modelo de datos de solicitud

        SaveConversationRequest.

    - Modelo de datos de respuesta

        SaveConversationResponse.

- **/conversation/{conversation_session_id}:**
    - Método

        GET

Internal



- Descripción

  Devuelve la conversación desde Azure Cosmos DB si esta existe.

- Modelo de datos de solicitud

  Enviar el conversation_session_id.

- Modelo de datos de respuesta

  GetConversationResponse.

- **/conversation/user/{user_id}:**
  - Método

    GET

  - Descripción

    Devuelve todas las conversaciones guardadas del usuario dado desde Azure Cosmos DB.

  - Modelo de datos de solicitud

    Enviar el user_id.

  - Modelo de datos de respuesta

    GetAllConversationsFromUserIDResponse.

- **/conversation/{conversation_session_id}:**
  - Método

    DELETE

  - Descripción

    Elimina la conversación desde Azure Cosmos DB si esta existe.

Internal

- Modelo de datos de solicitud

    Enviar el conversation_session_id.

- **/conversation/{conversation_session_id}:**
    - Método

        PUT

    - Descripción

        Actualiza una conversación en Cosmos DB.

    - Modelo de datos de solicitud

        UpdateConversationRequest

    - Modelo de datos de respuesta

        SaveConversationResponse

D-JDK-291

El presente apartado describe la arquitectura, componentes y funcionamiento del Frontend del sistema IA Sentencias Judiciales, una aplicación web desarrollada con **Java** y el framework **PrimeFaces**. Este sistema está diseñado para asistir a profesionales del derecho en la consulta, análisis y resumen de sentencias judiciales emitidas en los últimos 10 años, con el objetivo de facilitar la toma de decisiones jurídicas fundamentadas y eficientes.

El FrontEnd actúa como la interfaz de interacción entre los usuarios (abogados) y el motor de inteligencia artificial que procesa y resume las sentencias. A través de una interfaz intuitiva, moderna y responsiva, los usuarios pueden realizar búsquedas, visualizar resúmenes automáticos, acceder a estadísticas relevantes y consultar la evolución jurisprudencial de temas específicos.

Lenguaje y tecnologías principales

El FrontEnd del sistema IA Sentencias Judiciales está construido sobre la pila tecnológica de **Java EE** utilizando **Java Server Faces (JSF)** como motor de vistas y **PrimeFaces** como biblioteca de componentes UI. Esta arquitectura permite una separación clara entre la lógica de presentación, la lógica de negocio y el acceso a datos, facilitando el mantenimiento y la escalabilidad del sistema.

Tecnologías Utilizadas

- **Java EE**: Plataforma base para el desarrollo de aplicaciones empresariales.
- **JSF (JavaServer Faces)**: Framework MVC para la construcción de interfaces web basadas en componentes.
- **PrimeFaces**: Biblioteca de componentes UI rica y responsiva para JSF.
- **Facelets**: Motor de plantillas para JSF que permite una composición de vistas más limpia y reutilizable.
- **AJAX**: Utilizado a través de PrimeFaces para mejorar la experiencia del usuario sin recargar la página.

Repositorio de código

El código fuente del FrontEnd del Asistente Virtual Jurídico se encuentra alojado en un repositorio Azure DevOps, lo que permite el control de versiones, colaboración entre desarrolladores y trazabilidad de cambios.

- D-JDK-291 Azure DevOps Repo: https://AXA-MX-TECNOL-INFORMACION@dev.azure.com/AXA-MX-TECNOL-INFORMACION/D-JDK-291/_git/D-JDK-291

URL de despliegue

El sistema cuenta con diferentes ambientes para facilitar el desarrollo, pruebas y despliegue en producción. A continuación, se listan las URLs correspondientes a cada entorno:

Internal

Ambiente	URL de acceso	Descripción
Desarrollo	http://127.0.0.1:8080/login.xhtml	Entorno para pruebas internas
QA / Testing	https://sentenciasjudiciales.preprod.axa.com.mx/login.xhtml	Entorno para validación funcional
Producción	https://sentenciasjudiciales.axa.com.mx/	Entorno en vivo para usuarios finales

El acceso a los entornos puede requerir autenticación. Solicita credenciales al equipo de infraestructura si es necesario.

Headers

Derivado de una observación por parte del equipo de seguridad se sugirió la implementación de cabeceras de seguridad en la aplicación web desarrollada por AXA, en respuesta a las recomendaciones surgidas durante un proceso de prueba de penetración (pentest). Durante dicha evaluación, se identificaron varias vulnerabilidades relacionadas con la protección contra ataques comunes, como Cross-Site Scripting (XSS) y otros exploits que pueden afectar la integridad y confidencialidad de la aplicación.

Para mitigar estos riesgos, se decidió incorporar cabeceras de seguridad HTTP que refuercen la protección del entorno de ejecución y controlen el comportamiento del navegador frente a contenidos potencialmente peligrosos. La implementación se realizó mediante la creación de un filtro en la capa de la aplicación, que intercepta las respuestas HTTP y agrega las cabeceras necesarias antes de enviarlas al cliente.

Este proceso incluyó la definición y adición de varias cabeceras clave, tales como X-XSS-Protection, X-Content-Type-Options, X-Frame-Options y Strict-Transport-Security. La configuración de estas cabeceras fue ajustada para balancear la seguridad y la funcionalidad de la aplicación, asegurando un entorno más seguro para los usuarios y cumpliendo con las recomendaciones de las evaluaciones de seguridad realizadas.

Internal

Configuración y ejecución local

Esta sección describe los requisitos y pasos necesarios para configurar el entorno de desarrollo del asistente virtual IA-Sentencias Judiciales, tanto para los servicios de back-end desarrollados con Python como para el front-end desarrollado con Java EE, JSF y PrimeFaces.

Es importante considerar que el proyecto se encuentra estructura con las siguientes ramas:

- Ramas principales:
  - **main:** Rama estable para producción.
  - **dev:** Rama de desarrollo activo.
  - **qa:** Rama para pruebas antes de producción

- Ramas para el desarrollo:
  - **feature/*:** Ramas para nuevas funcionalidades.
  - **hotfix/*:** Ramas para correcciones en funcionalidades existentes.

Hay que asegurar que se sigue la convención de ramas y realizar pull requests para integrar cambios a develop o main.

SRV-PYT-788

Requisitos

La ejecución de este proyecto requiere la solicitud e instalación de:

- Python 3.12.2
- Pip 24.0 (Usualmente incluído con la instalación de Python)
- Docker Desktop >= 4.4.3
- Docker Engine >= 20.10.12 (Usualmente incluído con la instalación de Docker Desktop)

Internal

- Permisos de administrador para ejecutar Docker Desktop

La solicitud de estas dependencias puede hacerse a través de Silva.

Para el uso de este servicio de forma local es necesario:

1. Clonar su repositorio asociado: SVR-AZOCR-788

2. Copiar y pegar el archivo **.env.example** en la misma carpeta donde se encuentra y renombrarlo como **.env**
   En él se definen todas la variables necesarias para el despliegue de la aplicación, basta con identificar aquellas que no tienen un valor asociado y asignarles uno. En la sección Variables de entorno se describe con mayor detalle cada una de ellas.

3. Dentro de la carpeta del proyecto, es decir, dentro de SVR-AZOCR-788 se deben instalar las librerías definidas en el archivo "pyproject.toml", lo cual se logra ejecutando el comando:
   poetry install

   Esto creará un ambiente .venv y un archivo poetry.lock

Herramientas y librerías utilizadas

Las principales dependencias definidas en el archivo "pyproject.toml" son:

- **fastapi:** librería utilizada para el desarrollo de la API.
- **langchain:** utilizada para simplificar la interacción con los modelos de lenguaje
- **pdfplumber y fitz:** facilita la manipulación de documentos PDF y permite la extracción la información detallada de un PDF, como texto y formas.
- **uvicorn:** permite la ejecución de aplicaciones. Se utiliza para ejecutar aplicaciones en FastAPI
- **tiktoken:** utilizado para contar tokens para modelos lingüisticos.

Internal

Pasos para iniciar el servicio

1. Generar la imagen de Docker asociada a la aplicación con el comando docker compose build desde la raíz del proyecto. Hay que verificar que la aplicación **Docker desktop** esté abierta.
2. Ejecutar la imagen construida con docker compose up
   Una vez ejecutado el servicio, este será desplegado en la dirección http://localhost:8788
3. Si se requiere revisar sus funcionalidades o interactuar con la aplicación, esto se puede hacer a través de Swagger UI dado que la aplicación fue desarrollada usando FastAPI. Esto ya fue especificado en la **Arquitectura general del sistema**

SRV-PYT-784

Requisitos

La ejecución de este proyecto requiere la solicitud e instalación de:

- Python 3.12.2
- Pip 24.0 (Usualmente incluído con la instalación de Python)
- Docker Desktop >= 4.4.3
- Docker Engine >= 20.10.12 (Usualmente incluído con la instalación de Docker Desktop)
- Permisos de administrador para ejecutar Docker Desktop

La solicitud de estas dependencias puede hacerse a través de Silva.

Para el uso de este servicio de forma local es necesario:

1. Clonar su repositorio asociado: SVR-PYT-784
2. Realizar la descarga de los modelos que se usan para hacer la censura de la información y la identificación de entidades. Para ello, desde la raíz del repositorio es necesario ejecutar los siguientes comandos:

Internal



install poetry==1.8.3

poetry install --only dev

3. Una vez que se han instalado las dependencias, se debe haber creado una carpeta llamada **.venv** en la raiz del proyecto. El siguiente paso es realizar la descarga de los modelos ejecutando el siguiente comando:

   poetry run python setup.py

   El proceso de descarga puede tomar varios minutos, pero una vez completado, se debe haber creado una carpeta llamada **data** que contiene una carpeta llamada **models**, y que a su vez contiene los modelos necesarios

4. Copiar y pegar el archivo ".env.example" en la misma carpeta donde se encuentra y renombrarlo como ".env".

   En él se definen todas la variables necesarias para el despliegue de la aplicación, basta con identificar aquellas que no tienen un valor asociado y asignarles uno. En la sección **Variables de entorno** se describe con mayor detalle cada una de ellas.

5. Dentro de la carpeta del proyecto, es decir, dentro de SVR-PYT-784 se deben instalar las librerías definidas en el archivo “pyproject.toml”, lo cual se logra ejecutando el comando:
   poetry install

   Esto creará un ambiente .venv y un archivo poetry.lock

Herramientas y librerías utilizadas

Las principales dependencias definidas en el archivo “pyproject.toml” son:

- **fastapi:** librería utilizada para el desarrollo de la API.
- **langchain:** utilizada para simplificar la interacción con los modelos de lenguaje

Internal

- **spacy:** herramienta de procesamiento de lenguaje natural (NLP) diseñada para facilitar el reconocimiento de entidades nombradas.
- **uvicorn:** permite la ejecución de aplicaciones. Se utiliza para ejecutar aplicaciones en FastAPI
- **websockets:** permite la creación de aplicaciones que requieren una comunicación bidireccional y en tiempo real entre el cliente y el servidor.
- **pillow**: librería de procesamiento de imágenes que permite abrir, manipular y guardar imágenes en varios formatos. Es usada para la creación de la versión despersonalizada de la sentencia judicial.

Pasos para iniciar el servicio

1. Generar la imagen de Docker asociada a la aplicación con el comando docker compose build desde la raíz del proyecto. Hay que verificar que la aplicación **Docker desktop** esté abierta.
2. Ejecutar la imagen construida con docker compose up
   Una vez ejecutado el servicio, este será desplegado en la dirección http://localhost:8784
3. Si se requiere revisar sus funcionalidades o interactuar con la aplicación, esto se puede hacer a través de Swagger UI dado que la aplicación fue desarrollada usando FastAPI. Esto ya fue especificado en la **Arquitectura general del sistema**

Si al ejecutar las evaluaciones o ejecutar alguna prueba local se encuentra con algún **conflicto derivado del uso de tensorflow**, ejecute la siguiente línea de código desde la raíz del proyecto:

```
poetry add tensorflow tf-keras --group dev
```

Esto modificará el archivo pyproject.toml, **no suba este cambio al repositorio, pues es un problema que se ha encontrado de manera aislada.**

Internal

SRV-PYT-783

Requisitos

La ejecución de este proyecto requiere la solicitud e instalación de:

- Python 3.12.2
- Pip 24.0 (Usualmente incluido con la instalación de Python)
- Docker Desktop >= 4.4.3
- Docker Engine >= 20.10.12 (Usualmente incluido con la instalación de Docker Desktop)
- Permisos de administrador para ejecutar Docker Desktop

La solicitud de estas dependencias puede hacerse a través de Silva.

Adicionalmente, para el funcionamiento de este servicio es necesario contar con los siguientes componentes:

- SRV-PYT-784: Despersonalización de documentos
- SRV-AZOCR-788: Motor de reconocimiento óptico de caracteres

Las instrucciones para su despliegue fueron ya descritas.

Además, la aplicación hace uso de los servicios de Inteligencia Artificial y Almacenamiento siguientes:

- SecureGPT: Como proveedor de Modelos Largos de Lenguaje (LLM) y Modelos de Incrustaciones (Embeddings). Por lo que es necesario contar con acceso a Secure GPT model hub. Para realizar tu proceso de onboarding puedes dar clic aquí .
- AzureAISearch: Como base de conocimientos (base de datos vectorial).
- AzureCosmos DB: Como base de datos No-SQL para la adminisración de las conversaciones.

Para el uso de este servicio de forma local es necesario:

1. Clonar su repositorio asociado: SVR-PYT-783

Internal

2. Copiar y pegar el archivo **.env.example** en la misma carpeta donde se encuentra y renombrarlo como **.env**
   En él se definen todas la variables necesarias para el despliegue de la aplicación, basta con identificar aquellas que no tienen un valor asociado y asignarles uno. En la sección **Variables de entorno** se describe con mayor detalle cada una de ellas.

3. Dentro de la carpeta del proyecto, es decir, dentro de SVR-PYT-783 se deben instalar las librerías definidas en el archivo "pyproject.toml", lo cual se logra ejecutando el comando:
   poetry install

   Esto creará un ambiente .venv y un archivo poetry.lock

Herramientas y librerías utilizadas

Las principales dependencias definidas en el archivo "pyproject.toml" son:

- **fastapi:** librería utilizada para el desarrollo de la API.
- **langchain:** utilizada para simplificar la interacción con los modelos de lenguaje
- **pdfplumber y fitz:** facilita la manipulación de documentos PDF y permite la extracción la información detallada de un PDF, como texto y formas.
- **uvicorn:** permite la ejecución de aplicaciones. Se utiliza para ejecutar aplicaciones en FastAPI
- **azure-identity:** utilizada para la autenticación de los servicios de Azure a través del SPN (Service Principal).
- **azure-mgmt-Cosmos DB:** para la autenticación del servicio Azure Cosmos DB for Mongo a través del SPN.
- **azure-search-documents:** permite la interacción con el servicio Azure AI Search.
- **azure-cosmos:** permite la interacción con el servicio Azure Cosmos DB for Mongo.
- **pymongo:** para la interacción con el servicio Azure Cosmos DB for Mongo.
- **pypandoc-binary:** es un paquete que proporciona un wrappers para Pandoc, una herramienta universal de conversión de documentos.

Internal

- **weasyprint:** permite la conversión de HTML y CSS a PDF, es decir, se utiliza para generar documentos PDF a partir de contenido web.
- **websockets:** permite la creación de aplicaciones que requieren una comunicación bidireccional y en tiempo real entre el cliente y el servidor.
- **openai:** permite integrar capacidades de inteligencia artificial en las aplicaciones

Pasos para iniciar el servicio

4. Generar la imagen de Docker asociada a la aplicación con el comando docker compose build desde la raíz del proyecto. Hay que verificar que la aplicación **Docker desktop** esté abierta.
5. Ejecutar la imagen construida con docker compose up
   Una vez ejecutado el servicio, este será desplegado en la dirección http://localhost:8788
6. Si se requiere revisar sus funcionalidades o interactuar con la aplicación, esto se puede hacer a través de Swagger UI dado que la aplicación fue desarrollada usando FastAPI. Esto ya fue especificado en la **Arquitectura general del sistema**

SRV-PYT-785

Requisitos

La ejecución de este proyecto requiere la solicitud e instalación de:

- Python 3.12
- Docker Desktop >= 4.4.3
- Docker Engine >= 20.10.12 (Usualmente incluido con la instalación de Docker Desktop).
- Permisos de administrador para ejecutar Docker Desktop.
- Puede hacer la solicitud de estas dependencias a través de Silva.

Además, la aplicación hace uso de los servicios de Inteligencia Artificial y Almacenamiento siguientes:



- SecureGPT: Como proveedor de Modelos Largos de Lenguaje (LLM) y Modelos de Incrustaciones (Embeddings).
- AzureAISearch: Como base de conocimientos (base de datos vectorial).
- AzureCosmos DB: Como base de datos No-SQL para la adminisración de las conversaciones.

La aplicación es potenciada por inteligencia artificial, así que es necesario contar con acceso a Secure GPT model hub . Para realizar tu proceso de onboarding puedes dar clic aquí.

Para comenzar con el uso de la aplicación de forma local se requiere de los siguientes componentes:

1. Clonar el repositorio asociado al proyecto. SRV-PYT-785: Asistente Conversacional Chatbot Sentencias Judiciales
2. Copiar y pegar el archivo **.env.example** en la carpeta raíz del proyecto. Renombra el archivo como **.env**.

   En él se definen todas la variables necesarias para el despliegue de la aplicación, basta con identificar aquellas que no tienen un valor asociado y asignarles uno. En la sección **Variables de entorno** se describe con mayor detalle cada una de ellas.

Herramientas y librerías utilizadas

El desarrollo SRV-PYT-785 es un desarrollo en lenguaje Python, se trata de una API desarrollada en FastAPI y cuyas dependencias podemos encontrar en el archivo “poetry.toml” localizado en el repositorio.

Del grueso de las dependencias del proyecto, se destacan las siguientes:

- **fastapi**. Librería utilizada para el desarrollo de la API.
- **azure-identity.** Librería utilizada para la autenticación de los servicios de Azure a través del SPN (Service Principal).
- **azure-mgmt-Cosmos DB.** Librería utilizada para la autenticación del servicio Azure Cosmos DB for Mongo a través del SPN.

- **azure-search-documents.** Librería utilizada para la interacción con el servicio Azure AI Search.
- **azure-cosmos.** Librería utilizada para la interacción con el servicio Azure Cosmos DB for Mongo.
- **pymongo.** Librería utilizada para la interacción con el servicio Azure Cosmos DB for Mongo.

Pasos para iniciar el servicio

1. Genera la imagen de Docker asociada a la aplicación con el comando: docker compose build
2. Genera el contenedor Docker asociado a la aplicación con el comando: docker compose up
3. Las dependencias del aplicativo serán administradas por poetry lo que implica que el despliegue del aplicativo con el comando docker compose up creará un archivo de nombre "poetry.lock".
4. Interactuar con la aplicación.
    a. Cuando el contenedor de Docker ha mostrado el mensaje "Application Startup Complete", la aplicación será accesible a través de: http://localhost:8785
    Se te mostrará un mensaje de bienvenida.

    b. Para revisar las funcionalidades definidas en la aplicación, visita: http://localhost:8785/docs

    c. Para interactuar con las funcionalidades de la aplicación se requiere forzosamente de un "conversation_session_id", este ID se usa para identificar la conversación en Azure Cosmos DB, es por ello que es muy importante contar con uno.

Internal



d. Para obtener un "conversation_session_id", ejecuta una petición a la ruta "/conversation/", los datos de entrada de dicha funcionalidad son:

   i. "user_id": una cadena de texto que representa el ID de usuario.

   ii. "conversation_name": una cadena de texto que representa el nombre de la conversación.

e. Una vez que has obtenido el "conversation_session_id" podrás interactuar con el resto de las funcionalidades definidas en la aplicación.

D-JDK-291

Requisitos

La ejecución de este proyecto requiere la solicitud e instalación de:

- **Java JDK/Jakarta EE 17.0.1:** Entorno de ejecución y compilación de Java
- **Apache Maven 3.9.1:** Gestión de dependencias y construcción del proyecto
- **Visual Studio Code:** Entorno de desarrollo integrado compatible con Java EE
- **Servidor de Aplicaciones Tomcat:** Para desplegar la aplicación JSF
- **Navegador web:** Para pruebas y visualización del FrontEnd
- **Docker Desktop >= 4.4.3:** Contenedor del aplicativo
- **Docker Engine >= 20.10.12** (Usualmente incluído con la instalación de Docker Desktop)
- Permisos de administrador para ejecutar Docker Desktop

La solicitud de estas dependencias puede hacerse a través de Silva.

Para el uso de este servicio de forma local es necesario:

1. Clonar el repositorio asociado al proyecto: D-JDK-291

Internal

2. Copiar y pegar el archivo **.env.example** en la carpeta raíz del proyecto. Renombra el archivo como **.env**.

   En él se definen todas las variables necesarias para el despliegue de la aplicación, basta con identificar aquellas que no tienen un valor asociado y asignarles uno. En la sección **Variables de entorno** se describe con mayor detalle cada una de ellas.

Herramientas y librerías utilizadas

Dentro de las herramientas, existen una serie de dependencias que son necesarias para la ejecución del programa correctamente. A continuación, te mostramos estas dependencias:

Dependencias Principales

- jakarta.faces jakarta.faces-api 4.0.11 : API principal de JavaServer Faces
- org.primefaces primefaces 15.04 : Componentes UI ricos para JSF
- org.glassfish javax.faces 2.3.9 : Implementación de referencia de JSF
- jakarta.platform jakarta.jakartaee-web-api 10.0.0 :API para manejo de servlets

Integración con IA y Backend

- Jakarta.servlet.jsp.jstl jakarta.servlet.jsp.jstl-api 2.0.0 :Serialización y deserialización JSON
- Jakarta.el jakarta.el-api 4.5.13 : Integración

Seguridad y Autenticación

- org.apache.logging.log4j log4j-jul 2.24.3 : Para implementación del login
- org.apache.logging.log4j log4j-core 2.24.3: Para implementación del login

Testing y Desarrollo

Internal

- org.junit Junit-bom 5.11.0 : Framework de pruebas unitarias
- org.mockito mockito-core 5.14.0: Simulación de dependencias para pruebas

Las versiones pueden variar según el entorno. Se recomienda mantenerlas sincronizadas con el archivo .env y el entorno de despliegue.

Archivo pom.xml

Todas estas dependencias están declaradas en el archivo raíz del proyecto:

/PR-Axa-Sentencias/D-JDK-291/pom.xml, el cual también gestiona:

- Plugins de compilación y empaquetado.
- Configuración del perfil de desarrollo y producción.
- Integración con Docker y GitVersion.

Pasos para iniciar el servicio

1. Genera la imagen de Docker asociada a la aplicación con el comando: docker compose build
2. Genera el contenedor Docker asociado a la aplicación con el comando: docker compose up
3. En una terminal aparte con permisos de administrador, deberas ejecutar el siguiente comando: mvn clean install
4. Una vez concluido el proceso de despliegue, podrás visualizar los recursos en la ruta: http://127.0.0.1:8080/login.xhtml

Internal

Variables de entorno

Todos los servicios, a excepción del front-end (D-JDK-291), cuentan con las siguientes variables de entorno:

i. **APP_ENVIRONMENT:** Se trata del nivel del proyecto, es decir, *dev* para el desarrollo, *qa* para la versión de pruebas y *prod* para el ambiente productivo. Debe tomar el mismo valor en todos los servicios, según el ambiente en el que se encuentre el proyecto.

ii. Propiedades de loggers

   a. **LOGGER_NAME:** Nombre del logger.

   b. **LOGGING_LEVEL:** Nivel de logger. Puede tomar el valor DEBUG o INFO. En la sección **Logs de procesos exitosos y fallidos** se explica el uso de cada valor.

Además, por servicio se requieren las que a continuación se enlistan

SRV-PYT-788

i. **MODEL_NAME:** Modelo de embeddings utilizado para la tokenización de los textos

Internal

SRV-PYT-784

Únicamente necesita las 3 variables que todos los servicios deben tener.

SRV-PYT-783

Derivado de las conexiones a los servicios de Azure y a Secure GPT, el desarrollo se despliega con las siguientes variables de entorno:

i. SecureGPT:

- **AXA_SECURE_GPT_CLIENT_ID**: El ID de cliente que autentica el consumo de la API de SecureGPT. Se obtiene en el proceso de on boarding.
- **AXA_SECURE_GPT_CLIENT_SECRET**: La llave secreta asociada al ID de cliente que autentica el consumo de la API de SecureGPT. Se obtiene en el proceso de on boarding.
- **AXA_SECURE_GPT_ONE_ACCOUNT_URL:** El endpoint de One Account para la obtención de los tokens de autorización de las peticiones hacia SecureGPT.
- **AXA_SECURE_GPT_BASE_ENDPOINT:** El endpoint de SecureGPT. Hay uno para cada ambiente, “dev”, “preprod”, “prod”.
- **AXA_SECURE_GPT_OPENAI_API_VERSION**: La versión de la API de Azure Open AI para consumir los modelos de IA.
- **AXA_SECURE_GPT_EMBEDDINGS_MODEL_ID**: El ID del modelo de embeddings en SecureGPT que se usa para vectorizar el texto.
- **AZURE_OPENAI_EMBEDDINGS_DIMENSION:** La dimensión del embedding.
- **AXA_SECURE_GPT_LLM_ID:** El modelo largo de lenguaje en secureGPT

Internal

- **AXA_SECURE_GPT_OPENAI_BASE_EMBEDDINGS_ENDPOINT:** El endpoint del modelo de embeddings en SecureGPT.
- **AXA_SECURE_GPT_OPENAI_BASE_LLM_ENDPOINT:** El endpoint del modelo largo de lenguaje en SecureGPT.

Para mayor información sobre la API de SecureGPT puedes visitar: SecureGPT

ii. Azure AI Search:

- **AZURE_AI_SEARCH_ENDPOINT**: El endpoint asociado al recurso de Azure AI Search.

Esta variable puede obtenerse en el portal de Azure o contactando al administrador de los recursos para que te sea proporcionada. Es importante aclarar que el desarrollo local se logró con el uso de un sandbox, de esta forma, si el sandbox ha sido eliminado, será necesario crear uno nuevo.

- **AZURE_AI_SEARCH_INDEX_NAME:** El nombre del índice de Azure AI Search en donde se guardan los extractos de texto y sus vectores.

iii. Azure Cosmos DB:

- **AZURE_COSMOSDB_SUBSCRIPTION_ID:** El ID de suscripción al servicio de Azure Cosmos DB for Mongo.
- **AZURE_COSMOS DB_RESOURCES_GROUP_NAME:** El nombre del grupo de recursos de Azure donde se aloja Azure Cosmos DB.
- **AZURE_COSMOS DB_ACCOUNT_NAME:** Nombre del recurso Azure Cosmos DB for Mongo.

Internal



- **AZURE_COSMOSDB_DATABASE:** Nombre de la base de datos de Azure CosmosDB.
- **AZURE_COSMOSDB_CONTAINER:** El nombre del contenedor de Azure Cosmos DB for Mongo que alberga los **metadatos** de las sentencias
- **SUMMARY_AZURE_COSMOSDB_CONTAINER:** El nombre del contenedor de Azure Cosmos DB for Mongo que alberga los resúmenes de las sentencias

Algunas de estas variables pueden obtenerse en el portal de Azure o contactando al administrador de los recursos para que te sea proporcionada. Es importante aclarar que el desarrollo local se logró con el uso de un sandbox, de esta forma, si el sandbox ha sido eliminado, será necesario crear uno nuevo.

iv. Blob Storage

- **AZURE_BLOB_STORAGE_ENDPOINT:** Endpoint que permite la conexión con el recurso de Blob Storage

v. Service Principal SPN:

- **AZURE_CLIENT_ID:** El ID de client asociado al SPN.
- **AZURE_CLIENT_SECRET:** La llave de autorización asociada al SPN.
- **AZURE_TENANT_ID:** El ID del tenant asociado al SPN.

Estas variables deben ser solicitadas con el administrador de los recursos, además, el SPN debe configurarse para que los recursos asociados al aplicativo, es decir, Azure AI Search y Azure Cosmos DB for Mongo, cuente con los permisos de lectura y escritura pertinentes. Es importante aclarar que el

Internal

desarrollo local se logró con el uso de un sandbox, de esta forma, si el sandbox ha sido eliminado, será necesario crear uno nuevo.

vi. Para la conexión con los servicios de OCR y despersonalización

- **API_OCR_URL**: La URL asociada a la aplicación que permite extraer la información de un documento mediante la aplicación SRV-AZOCR-788: Motor de reconocimiento óptico de caracteres
- **API_CEN_URL**: La URL asociada a la aplicación que permite despersonalizar la información extraída de un documento mediante la aplicación SRV-PYT-784: Despersonalización de documentos
- **API_CEN_URL:** Es la versión con WebSockets de **API_CEN_URL**

SRV-PYT-785

i. Variables de entorno para el servicio Azure Cosmos DB:

Además de las definidas en el servicio SRV-PYT-783 también requiere:

- **AZURE_COSMOSDB_CONVERSATIONS_DATABASE:** El nombre de la base de datos de Azure Cosmos DB. Es el mismo que **AZURE_COSMOSDB_DATABASE en el servicio SRV-PYT-783**
- AZURE_COSMOSDB_CONVERSATIONS_CONTAINER: El nombre del contenedor/colección de Azure Cosmos DB dónde serán guardadas las conversaciones.

Internal

- **AZURE_COSMOSDB_DOCUMENT_SUMMARY_CONTAINER:** Toma el mismo valor que la variable **SUMMARY_AZURE_COSMOSDB_CONTAINER** en el servicio 783
- **AZURE_COSMOSDB_TEMP_CONVERSATIONS_TTL:** El tiempo de vida de las conversaciones que NO han sido guardadas. En segundos.

ii. Las variables definidas en SRV-PYT-783 para Azure AI Search

iii. Las variables definidas en SRV-PYT-783 para SPN

iv. AXA Secure GPT

Además de las definidas en el servicio SRV-PYT-783 también requiere:

- **AZURE_OPENAI_EMBEDDINGS_MODEL_NAME:** El nombre real del modelo de embddings.

v. RAG

- **API_DOCS_URL:** La URL del servicio de administración de Documentos. Esta se utiliza para que las citas a los documentos puedan descargar la versión censurada de este.

D-JDK-291

Las variables de este servicio son totalmente independientes a las descritas anteriormente, a excepción de la definición de **APP_ENVIRONMENT** que también es necesaria.

Internal

i. Variables de configuración de APIs:

- **API_RAG_URL:** Se trata del endpoint del RAG para el asistente virtual.
- **API_DOCS_URL:** Se trata del endpoint de admnistración de documentos
- **WS_API_RAG_URL:** Web Service del endpoint de Documentación.

iii. Variables de OneAccount:

- **ONE_ACCOUNT_CLIENT_ID:** Client ID que solicita el servicio de OneAccount.
- **ONE_ACCOUNT_CLIENT_SECRET:** Client Secret que solicita el servicio de OneAccount.
- **ONE_ACCOUNT_CONFIG_URL:** Llamado página principal del OneAccount.
- **ONE_ACCOUNT_CALLBACK_URL:** Para obtener los elementos necesarios del api de OneAccount.

Internal

Ingesta y procesamiento de documentos

Tanto la ingesta de documentos como su procesamiento se realizan a través del servicio SRV-PYT-783, el cual funge como servicio orquestador del tratamiento del documento. Específicamente, el proceso para el que está programado este servicio, y que el usuario final puede percibir es el siguiente:

1. Carga de la sentencia judicial a la herramienta IA-Sentencias Judiciales.
2. Verificación del formato del archivo para determinar si se trata de un PDF; en caso contrario, se realiza la conversión correspondiente.
3. Análisis página por página para identificar la presencia de imágenes y por tanto clasificarla como nativa o no nativa.
4. Mediante una llamada al servicio SVR-PYT-788 se extrae el texto correspondiente a cada página, dicha extracción depende del análisis del punto 3.
5. Del texto extraído por página, se aplica la primera capa de despersonalización con modelos de Spacy haciendo una llamada al servicio SRV-PYT-784
6. Aplicación de la segunda capa de despersonalización realizando llamadas a secureGPT
7. Generación de resúmenes del contenido del documento. Estos se guardan en Cosmos y ayudan al asistente a tener un contexto de la sentencia completa cuando el usuario realiza una pregunta
8. Extracción de entidades relevantes realizando también una llamada a secureGPT. Estas entidades son el magistrado ponente, el circuito o localidad, el tribunal (materia y número), la fecha de sentencia y la resolución de la misma.
9. Almacenamiento en AI-Search de los vectores generados
10. Almacenamiento en Cosmos de los metadatos y del resumen generado
11. Creación y almacenamiento el Blob Storage de la sentencia PDF despersonalizada

Internal

Criterios de aceptación de documentos

El proyecto IA-Sentencias Judiciales está totalmente desarrollado tomando como referencia la estructura propia de una sentencia judicial, es decir, con base en este tipo de documentos se diseñó el procesamiento. En consecuencia, aunque otro tipo de documentos pudieran procesarse sin errores técnicos, la ausencia de dicha estructura provocaría fallos en la extracción de metadatos y generaría sesgos en las respuestas del asistente. Por lo anterior, los documentos que se carguen para actualizar la base de conocimiento deben ser exclusivamente **sentencias judiciales** que contengan la siguiente estructura: **encabezado, resultandos, considerandos y resolutivos.**

Formatos admitidos

Para lograr respuestas óptimas en el asistente conversacional es importante mantener actualizada la información de la base de conocimiento.

Dicha base es capaz de procesar toda aquella sentencia con alguna de las siguientes extensiones:

- .pdf
- .docx
- .jpg
- .png
- .jpeg

Esto es posible ya que el endpoint **/document/convert-to-pdf/** del servicio de SRV-PYT-783 realiza una conversión de cualquiera de este tipo de documentos a PDF para posteriormente poder extraerle la información.

Cabe mencionar que aun cuando se cargue un documento en una extensión distinta a PDF, el documento despersonalizado se generará y guardará en PDF.

Internal

Sobre la orientación del documento

Para garantizar una correcta lectura y análisis de la información, se sugiere que los documentos cargados estén **orientados verticalmente**. Otras orientaciones pueden afectar la precisión de los resultados y/o tiempos de procesamiento.

Sobre el idioma

Si bien los modelos de lenguaje cuante con la capacidad de comprender múltiples idiomas, el desarrollo de la herramienta –incluyendo la definición de prompts, el ajuste de parámetros y la configuración de reglas de negocio- se realizó en español. Por esta razón, para garantizar la mayor precisión y consistencia en los resultados, se recomienda que los documentos cargados se encuentren redactados es **español**.

Sobre el nombre del documento

El proceso de despersonalización únicamente garantiza dicha censura en el contenido del documento. Por tal razón, es importante que, si se identifica algún dato sensible en el nombre de la sentencia, entonces éste sea cambiado ya que la herramienta no podrá censurar dicho valor y entonces una persona si pudiera ser rastreada.

Por tanto, en el nombre del documento no debe haber algún dato que pueda asociarse a una persona, tampoco debe contener caracteres especiales.

Sobre el tamaño

Según los recursos sobre los cuales la aplicación se esté ejecutando, dependerá la rapidez y la prevención de errores. En este sentido el tamaño máximo por sentencia judicial a cargar es de **50MB.**

Gestión y versión de documentos durante la carga

Esta sección busca aclarar el esquema de versionado de los documentos, es decir, precisar en qué casos un archivo puede ser procesado por la herramienta IA-Sentencias Judiciales, garantizando que no haya sido cargado previamente.

Internal

Cada documento tiene un ID único asociado, el cual se toma a partir del HASH del mismo. Al momento de la carga se verifica que este valor no exista en la base de conocimiento y para ello hay que tomar en cuenta lo siguiente:

- **Si el nombre y el contenido de dos documentos es distinto**, entonces el HASH de ambos es distinto y por tanto **procesa y guarda ambos** documentos.

- Si los **nombres son iguales y el contenido es distinto**, el HASH también será distinto pues éste se basa no solo en el nombre del documento, sino también en el contenido y sus metadatos asociados, como el momento de creación del documento. Por lo que procesará y guardará ambos documentos.

  Hay que tener cuidado en este caso, pues dos documentos distintos pero que comparten el nombre pueden confundir tanto al asistente como al usuario.

- Si **el nombre y el contenido son iguales**, es decir, se trata del mismo documento, entonces **NO se vuelve a procesar** y en la plataforma se verá rápidamente que el documento ha sido cargado. Esto además indica que el documento no se reescribe.

- Si el **nombre es distinto y el contenido es igual**, hay 2 opciones dependiendo de a qué se le llame como “el mismo documento”:

    - Si es literalmente el mismo contenido, pero solo se le cambió el nombre, entonces no lo volverá a procesar y también en la plataforma se verá que rápidamente aparece que el documento se procesó, pero en realidad el documento no se reescribe y se queda el que ya estaba cargado.
    - Si tiene el mismo contenido, pero se volvió a crear el PDF ya no cuenta como el mismo documento, cuenta como el punto b, pues aun cuando tenga el mismo contenido, el hash y entonces si se procesará como un documento nuevo y se guardará.

Internal

Despersonalización (Capas de censura con Spacy y LLM)

Una vez extraído el texto original de la sentencia judicial, es de suma importancia despersonalizar la información para garantizar que ningún dato sensible pueda ser asociado a una persona física o moral.

En este sentido, el proceso se lleva a cabo en dos capas:

Modelos de Spacy

La primera se realiza mediante el servicio SRV-PYT-784, el cual mediante el modelo es_core_news_lg-3.8.0 permite asociar a cada palabra un tipo de entidad que él mismo reconoce: [PERSON] para nombre propios, [ORGANIZATION] para empresas y organizaciones, [ID] para identificadores como el RFC, [EMAIL] para correros y [POLICY] referenciando números de pólizas o contratos, [GPE] para entidades Geopolíticas, es decir países y ciudades, [LOC] para lugares que no son GPE, [DATE] para entidades de tiempo y cantidad como Fechas , [TIME] para tiempo y [MONEY] para dinero.

Después, para cada entidad reconocida se construye un diccionario con la palabra reconocida, el tipo de entidad asociada, si debe censurarse, y las posiciones que esas palabras toman en el texto original. Además de la versión despersonalizada del texto original.

Por ejemplo, del texto:

```
"El señor Antonio Gómez ha comprado una casa con ayuda de un préstamo
                        bancario el 25 de enero"
```

Y definiendo las entidades que se quieren censurar:

```
{ "type": "PER", "replace_with": "[PERSON]", "collapse": false },
```

Se obtiene un diccionario por entidad reconocida como el siguiente:

```
{    "text": "Antonio Gómez",
      "type": "PER",
      "censoring": true,
      "custom": false,
```


Internal



```
        "words": [2, 3]
    },
    {    "text": "25 de enero",
         "type": "DATE",
         "censoring": false,
         "custom": false,
         "words": [15, 16, 17]
    }
```

Y un texto despersonalizado:

```
"El señor [PERSON] [PERSON] ha comprado una casa con ayuda de un préstamo
                        bancario el 25 de enero"
```

Hay que notar que dado que solo se ha definido que sea la entidad “PER” la que se censure, el modelo de Spacy identifica más entidades, pero no se censuran.

Modelo Largo de Lenguaje (LLM y secure-GPT)

La segunda capa de despersonalización se realiza usando secureGPT para detectar y reemplazar entidades sensibles que en la primera capa no hayan sido detectadas.

Un modelo largo de lenguaje no es un modelo determinista, es decir, para cada consulta, aún cuando la pregunta sea escrita de la misma forma, el modelo solo conservará la idea de la respuesta, pero esta no será devuelta con el mismo formato. Por dicha razón, se han implementado dos aproximaciones verificando que se conserven las posiciones de las palabras y lograr una correcta censura, entonces en caso de que la primera falle, se ejecutará la segunda. Para ambos casos, los prompts utilizados se encuentran en el directorio **app\api\core\llm\censor.py**

- **Primera aproximación en donde el LLM devuelve el texto censurado con las entidades etiquetadas**

En este se busca que un texto salga tal cual fue metido, pero con las entidades censuradas. Es decir, busca replicar el comportamiento de Spacy. Para ello, primero se define el “rol” que el LLM asumirá:

Internal

```
system = "Eres un experto en identificación de información y datos sensibles o personales en textos judiciales"
```

En el *prompt* primero se da una visión general de lo que se busca realizar y se define aquello que se considera como un dato sensible, es decir, nombres propios e identificadores que se pueden asociar a una persona y que por tanto no deben mostrarse:

```
Tu tarea es identificar y sustituir los datos sensibles en el texto judicial dado, para ello se define como datos sensibles la siguiente información:

- Cada nombre propio y apellido de personas físicas, donde el nombre propio es cualquier nombre de pila (ejemplos: "Luis", "Gabriela", "Uriel"), el apellido es cualquier apellido (ejemplos: "Gómez", "Martínez", "Gómez")

- Cada identificador de pólizas. Pueden ser valores numéricos o alfanuméricos. Ejemplo: 1345265478034

- Cada identificador de contratos. Pueden ser valores numéricos o alfanuméricos.

- Cada identificador de números de tarjetas bancarias de crédito y débito.

- Cada placa de automóviles o motocicletas. Pueden ser valores numéricos o alfanuméricos. Ejemplo: ABC-123-A o ABC-123 para vehículos, y A12-BC o 12-ABC para motocicletas.

- Cada dirección de domicilios particulares. Ejemplo: Av. Mixocac #37, Ciudad de México 01800.

- Cada RFC o Registro Federal de Contribuyentes de personas morales, entendiéndolo como 12 caracteres en donde los 3 primeros valores corresponden a las letras iniciales de la empresa o una combinación de éstas; los siguientes seis caracteres son los dos últimos dígitos del año (aa), mes (mm) y día (dd) de su constitución y los últimos tres dígitos corresponden a una homoclave. Ejemplo: RDL0904102F4

- Cada RFC o Registro Federal de Contribuyentes de personas físicas, entendiéndolo como 13 caracteres en donde las primeras dos letras son las del apellido paterno, después va la primera letra del apellido materno, luego la
```


Internal

```
primera letra del nombre, seguido de la fecha de nacimiento, los dos últimos dígitos del año (aa), mes (mm) y día (dd); los últimos 3 caracteres son una homoclave. Ejemplo: AMMA9809179L2

- Cada IMEI o Identidad Internacional de Equipo Móvil que es un número único de 15 dígitos que identifica a cada dispositivo móvil en el mundo

- Cada BIN o Bank Identification Number que son los primeros dígitos (generalmente 6) de una tarjeta que identifican al banco o institución financiera que la emitió, así como el tipo de tarjeta (crédito o débito)
```

Después, se definen las instrucciones para resolver la tarea acompañadas de ejemplos:

```
      # Instrucciones de censura

- Reemplaza cada nombre propio o apellido con **una instancia de la etiqueta [PERSON] sustituyendo palabra por palabra**, si un nombre tiene varias palabras, reemplaza cada una por un [PERSON]

      Ejemplos:
     'Miguel Angel Morelos Martínez' -> [PERSON] [PERSON] [PERSON] [PERSON]
     'Miguel' -> [PERSON]
     'Miguel Angel' -> [PERSON] [PERSON]
     'Martínez' -> [PERSON]
     'Miguel Morelos' -> [PERSON] [PERSON]
     'Miguel Angel Martínez' -> [PERSON] [PERSON] [PERSON]

- Reemplaza cada uno de los identificadores de pólizas por la etiqueta [POLICY]

      Ejemplos:
      'póliza número 111273912301' -> póliza número [POLICY]
      'póliza 333272915301' -> póliza [POLICY]

- Reemplaza cada uno de los identificadores de contratos por la etiqueta [POLICY]

      Ejemplo: 'contrato 456789' -> contrato [POLICY]

- Reemplaza cada uno de las placas de automóviles o motocicletas por la etiqueta [LICENSE_PLATE]
```


Internal

- Reemplaza cada valor que se identifique como IMEI, RFC, BIN o número de tarjeta bancaria como [POLICY]

- Reemplaza cada una de las direcciones de domicilios particulares por la etiqueta [ADDRESS].

Ejemplo:
Av. Mixocac #37, Ciudad de México 01800. -> [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS]

Se definen algunos ejemplos de la tarea completa:

- Reemplaza cada palabra que represente un dato sensible por la etiqueta correspondiente. Si no hay entidades por censurar devuelve el mismo texto de entrada.

Ejemplos:

Texto de Entrada 1:
El día 24 de diciembre de 2023, Miguel Angel Martínez Gómez con póliza número 111273912301 ha sido afectado por incumplimiento al no cubrir el siniestro sufrido en su automóvil con placas DEF-479-S. El contrato 456789 establece su domicilio en Av. Mixocac #37, Ciudad de México 01800.

Texto de Salida 1:
El día 24 de diciembre de 2023, [PERSON] [PERSON] [PERSON] [PERSON] con póliza número [POLICY] ha sido afectado por incumplimiento al no cubrir el siniestro sufrido en su automóvil con placas [LICENSE_PLATE]. El contrato [POLICY] establece su domicilio en [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS] [ADDRESS]

Texto de Entrada 2:
El día 24 de diciembre de 2023 se ha presentado un siniestro por robo total de automóvil.

Texto de Salida 2:
El día 24 de diciembre de 2023 se ha presentado un siniestro por robo total de automóvil.

Internal

```
      Texto de Entrada 3:
Esto es un texto corto sin contexto judicial que no tiene entidades por
censurar

      Texto de Salida 3:
Esto es un texto corto sin contexto judicial que no tiene entidades por
censurar
```

Finalmente, se define el formato de salida:

```
- Reescribe el texto sustituyendo los datos sensibles con su correspondiente
etiqueta.

- Si no hay entidades por censurar devuelve el mismo texto de entrada.

- Arroja el resultado como una cadena de texto con el mismo formato de entrada.

- La cantidad de palabras censuradas debe ser igual a la cantidad de palabras
originales censuradas.

- No elimines o añadas palabras.

- No des explicaciones ni mensajes adicionales.

- No incluyas el prefijo "Texto:"
```

- **Segunda aproximación en donde el LLM devuelve una lista con las nuevas entidades que deben censurarse.**

Esta aproximación se realiza cuando falla que el texto de entrada y de salida sea el mismo, pero reemplazando las entidades sensibles por alguna etiqueta, y entonces se busca crear una lista con las entidades que deben ser censuradas, para después buscar su posición en el texto original. Para ello, primero se define el “rol” que el LLM asumirá:

Internal

```
system = "Eres un experto en identificación de información y datos sensibles o personales en textos judiciales"
```

En el prompt primero se da una visión general de lo que se busca realizar:

```
Tu tarea es extraer las entidades del texto judicial dado y asociarle una etiqueta:

- Para la etiqueta "PER": Cada nombre propio y apellido de personas físicas, donde el nombre propio es cualquier nombre de pila (ejemplos: "Luis", "Gabriela", "Uriel"), el apellido es cualquier apellido (ejemplos: "Gómez","Martínez")

- Para la etiqueta "POLICY": Cada identificador de claves de pólizas y/o claves de contratos. Pueden ser valores numéricos o alfanuméricos. Ejemplos 345564, o póliza número 67357.

- Para la etiqueta "LICENSE_PLATE": Cada placa de automóviles o motocicletas. Pueden ser valores númericos o alfanuméricos. Ejemplo: ABC-123-A o ABC-123 para vehículos, y A12-BC o 12-ABC para motocicletas.

- Para la etiqueta "ADDRESS" Cada dirección de domicilios particulares. Ejemplo: Av. Mixocac #37, Ciudad de México 01800.

- Identifica cada dato sensible sin importar que esté escrito en solo mayúsculas, solo minúsculas o con ambas
```

Y se define el formato de salida:

```
- Un diccionario sin saltos de línea cuya clave es la entidad identificada respetando el formato del texto y el valor es la etiqueta asociada.
- Si una entidad se repite, devuelvela las veces que aparezca.
- No des explicaciones ni mensajes adicionales.
- No incluyas el prefijo "Texto:"
- No etiquetes fechas
- Etiqueta únicamente la información distinta de fechas y cantidades monetarias
- Identificadores de leyes (ejemplos: Ley de Amparo, Ley General de Instituciones y Sociedades Mutualistas de Seguros), artículos (ejemplo: art. 3) o instituciones gubernamentales (ejemplo: Suprema Corte de Justicia de la Nación) no deben estar etiquetadas
- Respeta el formato del texto. Por ejemplo:
```


Internal



{{"Maria Fernanda":"PER","MARIA Fernanda":"PER", "MARIA": "PER"}} son dos entidades distintas, devuelve cada una como parte del diccionario

Una vez censurado el texto, ya se pueden extraer entidades relevantes y generar resúmenes.

Proceso de extracción de metadatos

Los metadatos de una sentencia judicial son los datos extraídos, usando secureGPT, que se emplearán como filtros en la visualización de la información. En otras palabras, constituyen variables que facilitan la obtención de insights de la base de conocimiento.

Entre estos datos se encuentran:

- Magistrado ponente (Dict[str])

  Esta información, aunque se trata de un dato personal, no es censurada por lo que el proceso para su extracción sigue un proceso distinto, ya que a diferencia de todas las demás entidades [PERSON], el nombre del magistrado primero se asocia a una "etiqueta" que tendrá un nombre aleatorio.

  Más específicamente:

  - Con el endpoint **/censor/label/** se obtienen las etiquetas de todas las entidades del tipo "PER"

  - Con el endpoint **/censor/text/** se regenera el texto en su versión censurada, pero se especifica que toda aquella palabra reemplazada a [PERSON] se cambie a un nombre aleatorio y se crea un diccionario en donde la clave es el nombre aleatorio y el valor el nombre real: {"Juan": "Luis", "Pérez":"García"}

  - Se realiza la extracción de todas las entidades, y hasta este punto, el magistrado es un nombre aleatorio. **Esto garantiza que al LLM no se le ha pasado algún nombre real.**

Internal



- Dado que se ha construido un diccionario, se hace un match entre los nombres reales y ficticios para regresar entonces, en la entidad Magistrado ponente, el nombre real de este funcionario público.

- Estado o localidad (str)
- Circuito (str)
- Número y Materia del tribunal (Dict[str])
- Fecha de sentencia (str en formato “AAA-MM-DD")
- Resolución (str)

  Toma el valor **1 cuando Axa Seguros México ha perdido el caso.** Esta definición se debe a la importancia de conocer las causas de los casos perdidos y facilitar su conteo.

Como ya se mencionó, este proceso se realiza usando secureGPT, por lo que a continuación se describe el prompt utilizado que se encuentra en el directorio **\app\api\core\llm\extractions.py**. Para ello, primero se define el “rol” que el LLM asumirá:

```
system = "Eres un experto en abogacia"
```

Después se da una visión general de lo que se busca realizar y se define aquello que se necesita extraer:

```
Debes identificar y extraer los siguientes campos en texto judicial dado:

- Magistrado ponente: Es el magistrado (o juez/juzgador) encargado de redactar la sentencia y emitir la resolución dentro del tribunal. Suele ser designado como "ponente", "presidente" o "responsable de la resolución". Aunque otros jueces o magistrados pueden figurar en la sentencia, no participan en la redacción ni en la decisión, y su mención se limita a su rol como parte del tribunal.
```


Internal

*- **Magistrado ponente completo**: Es el mismo que el magistrado ponente*

*- **Circuito**: Generalmente, se identifica con un número, como "primero", "segundo", etc. Si no se utiliza esta forma, se hace referencia al estado y, en ocasiones, a la localidad donde se emite la sentencia. Aunque es posible mencionar más de un circuito, en este campo se debe registrar únicamente aquel donde se emite la sentencia, sin incluir circuitos que se citen como referencia.*

*- **Estado o localidad**: Es el estado o localidad donde se emite la sentencia*

*- **Tribunal**: Es el tribunal o juzgado donde se resuelve la sentencia. Consta de un número y materia. La materia puede ser civil, penal, mercantil, administrativa, laboral ,fiscal, etc.*

*- **Fecha de sentencia**: Es la fecha que define el inicio de la sentencia, es decir, cuando comienza su redacción.*

*- **Resolución** (1 si ganó la parte demandante, 0 si no)*

Después, se definen las instrucciones para resolver la tarea acompañadas de ejemplos:

- Si no se cumplen las definiciones dadas para cada campo, entonces se queda vacío.

- El número de tribunal no es el número de amparo directo.

- Una resolución se va a tomar de aquel contexto que te de suficiente información sobre si ganó el demandante o el demandado. Si en el texto dado no hay suficiente contexto deja el campo vacío.

- Magistrado ponente completo es un nombre propio nombre propio sin títulos como "Magistrado", "Lic.", "Licenciado, "Dr.", etc., solo el nombre propio completo

- El Magistrado ponente es el valor de Magistrado ponente, pero debe devolverse como un diccionario con la estructura: {{"nombre": "", "apellido_paterno": "", "apellido_materno": ""}}:

Internal

```
   * Cuando se tiene solo un nombre y un apellido entonces se asigna el
     apellido al campo "apellido_paterno" y se deja "apellido_materno"
     vacío
   * Si hay 2 apellidos, el primero es "apellido_paterno" y el segundo es
     "apellido_materno"
   * Los nombres (pueden ser uno o más de uno) se colocan en el campo
     "nombre"
   * No incluyas títulos como "Magistrado", "Lic.", "Dr.", etc., solo el nombre
completo
   * Si no hay nombre del magistrado deja los campos como vacío

   * Ejemplos:
    - {{"nombre":"Juan","apellido_paterno":"Pérez","apellido_materno": ""}}
 -    {{"nombre":    "Luis    Fernando",    "apellido_paterno":    "Ramírez",
"apellido_materno": "López"}}
```

Finalmente, se especifica el formato de salida:

```
-Responde solo con un diccionario que registre cada elemento sin saltos de
línea en el formato

-El Tribunal debe devolverse como un diccionario con la estructura: {{"número:
"", "materia":""}}

-Cuando se tenga solo el número, materia queda vacío y viceversa

Ejemplos:

* Primer Tribunal Colegiado en Materia Civil -> {{"número: "Primer Tribunal
Colegiado", "materia":"Civil"}}

*   Juzgado   quinto   de   lo   civil   ->   {{"número:   "Juzgado   quinto",
"materia":"Civil"}}
```

La obtención de estos metadatos se realiza partiendo el texto extraído en chunks de 8500 tokens, es decir y de cada uno de ellos extraer un diccionario del tipo:

```
{{"Magistrado ponente completo": "", "Magistrado ponente": "", "Circuito": "",
"Estado   o   localidad":   "",   "Tribunal":   "",   "Fecha   de   sentencia":   "",
"Resolución": ""}}
```


Internal

El cual se construye analizando cada página para extraer cada campo y una vez encontrado su valor, éste ya no se busca.

Después, se hace un barrido de los diccionarios construidos, tal que los metadatos que se sabe que se encuentran siempre en las primeras páginas, sean los que se devuelvan a partir del 1er diccionario construido.

Generación de resúmenes con LLM

Los resúmenes son creados a partir de la versión final de la información censurada y se toman como contexto para que el asistente pueda responder con mayor presición. Se almacenan en CosmosDB y también se obtienen usando secureGPT, por lo que a continuación se explica el prompt utilizado que se encuentra en el directorio **\app\api\core\llm\summary.py:**

Primero se define el "rol" que el LLM asumirá:

```
system = "Eres un abogado experto en derecho procesal civil, especialmente en seguros"
```

Después se da un contexto general de la tarea a realizar:

```
Estás leyendo una sentencia judicial donde el demandado es una aseguradora. Tu tarea es generar un resumen del texto proporcionado, considerando lo siguiente:
```

Se definen las instrucciones para realizar la tarea:

```
1. Si se continúa desde un resumen anterior, debes tenerlo en cuenta para mantener la coherencia.

2. Extrae las **causas por las que se gana o se pierde el caso** (de parte de la aseguradora).

3. Menciona detalladamente cuáles fueron **las pruebas presentadas o ausentes** que fueron determinantes.
```


Internal

```
4. Incluye **los artículos legales y considerandos** relevantes que fundamentan la decisión resolutiva y explica que tienen que ver con la resolución o en que ayudaron en ella.

5. Resume el contenido legal y fáctico de forma clara y concreta, manteniendo terminología jurídica apropiada.
```

Se define la instrucción que nos ayuda a garantizar una vez más que no haya algún dato sensible que pueda asociarse a una persona:

```
6. No incluyas en el resúmen:
      - nombres y apellidos de personas. Ejemplo: Samuel García López

      - nombres de organizaciones. Ejemplo: Alfa Seguros México

      - números de pólizas o números de contratos. Ejemplo: 1345265478034

      - números de tarjetas bancarias, de crédito y débito.

      - valores numéricos o alfanuméricos que representen placas de
      automóviles o motocicletas. Ejemplo: ABC-123-A o ABC-123 para
      vehículos, y A12-BC o 12-ABC para motocicletas.
      - direcciones particulares. Ejemplo: Av. Mixocac #37, Ciudad de México
01800.

      - RFC o Registro Federal de Contribuyentes de personas morales,
      entendiéndolo como 12 caracteres en donde los 3 primeros valores
      corresponden a las letras iniciales de la empresa o una combinación de
       éstas; los siguientes seis caracteres son los dos últimos dígitos del
       año (aa), mes (mm) y día (dd) de su constitución y los últimos tres
      dígitos corresponden a una homoclave. Ejemplo: RDL0904102F4

      - RFC o Registro Federal de Contribuyentes de personas físicas,
      entendiéndolo como 13 caracteres en donde las primeras dos letras son
      las del apellido paterno, después va la primera letra del apellido
      materno, luego la primera letra del nombre, seguido de la fecha de
      nacimiento, los dos últimos dígitos del año (aa), mes (mm) y día (dd);
       los últimos 3 caracteres son una homoclave. Ejemplo: AMMA9809179L2

      - IMEI o Identidad Internacional de Equipo Móvil que es un número único
de 15 dígitos que identifica a cada dispositivo móvil en el mundo
```


Internal

```
      - BIN o Bank Identification Number que son los primeros dígitos
      (generalmente 6) de una tarjeta que identifican al banco o institución
       financiera que la emitió, así como el tipo de tarjeta (crédito o
      débito)

7. Si el texto a resumir es muy corto (menos de 100 carcateres), devuelve el
mismo texto sin algún mensaje adicional

        {f"Resumen anterior: {prev_summary}" if prev_summary else ""}
        Texto a resumir: {chunk}
```

Se especifica el formato de salida:

```
- Si hay un resumen previo, devuelve únicamente el nuevo resumen coherente y
acumulativo, sin repetir lo ya resumido anteriormente y como un string sin
explicaciones ni mensajes adicionales.

- Si no hay resumen previo o el texto es muy corto, devuelve únicamente el
mismo texto original, sin tomar en cuenta ningún contexto dado y sin repetir
lo ya resumido anteriormente y como un string sin explicaciones ni mensajes
adicionales.
```

La generación de resumes se realiza a través de varias llamadas a secureGPT, esto se hace dado que la sentencia puede ser extensa. Entonces:

- Cuando el texto no fue partido en chunks por no ser tan grande, se resumen la sentencia judicial completa.
- Cuando se tiene más de un chunk, se resume el 1ro y éste se toma como contexto del 2do chunk para generar su propio resumen. Al final procesa todos los resumenes generados por chunk para obtener un resumen final.

Internal

Almacenamiento y BBDD

Estructura de almacenamiento

El proyecto PDP-767 almacena información relevante a las sentencias que se han procesado y a las conversaciones que los usuarios han desarrollado. De esta forma, se cuenta con los siguientes servicios de almacenamiento:

- **Azure Blob Storage.** En este servicio se guardarán las sentencias procesadas una vez que ya fueron censuradas y validadas por los usuarios.
- **Azure Comos DB for Mongo.** En este servicio guardaremos metadatos de las sentencias, resúmenes de estas y las conversaciones de los usuarios.
- **Azure AI Search.** Este es el servicio que funge como base de conocimientos, en él, se guardaron los contenidos de las sentencias, así como su vector embedido, lo que permite la ejecución de consultas vectoriales.

El desarrollo SRV-PYT-783 se estableció como el desarrollo encargado de la orquestación de los desarrollos SRV-PYT-784 (Censura de documentos) y SRV-AZOCR-788 (Extracción de información mediante tecnología OCR). Es por este hecho que el mismo servicio SRV-PYT-783 administra los documentos en los servicios de almacenamiento, en otras palabras, las funciones de guardado, actualización y eliminación de documentos serán encontradas en el código del desarrollo SRV-PYT-783.

Finalmente, y recordando que el desarrollo SRV-PYT-785 es el asistente conversacional, es en este dónde podremos encontrar las funciones de guardado, actualización y eliminación de conversaciones.

Blob

El servicio Azure Blob Storage es el encargado de almacenar los documentos una vez que estos fueron censurados, cada uno de estos archivos será guardado en un contenedor nombrado **“censored-documents”**.

Internal

Cosmos

El servicio Azure ComosDB for Mongo es el encargado de almacenar metadatos asociados a los documentos, resúmenes de estos mismos y las conversaciones de los usuarios. Todos estos objetos serán almacenados en la base de datos “legal-assistant”, sin embargo, serán guardados en su propia colección.

- Base de datos: “legal-assistant”:
    - Colección: “document-metadata”. En esta colección se guardan los metadatos asociados a los documentos que ya han sido procesados.
    - Colección: “document_summary”. En esta colección se guardan los resúmenes asociados a los documentos que ya han sido procesados.
    - Colección: “conversations”. En esta colección se guardan las conversaciones de los usuarios.

Azure AI-Search

El servicio Azure AI Search es el encargado de almacenar la base de conocimientos, en ella, cada uno de los documentos que ya ha fue procesado, se particiona en extractos de texto, luego, a cada uno de estos extractos se le asocia un vector embebido usando modelos de embeddings de SecureGPT, finalmente, este extracto de texto junto a su respectivo vector, se almacenan en Azure AI Search, específicamente en el índice “documents”.

Internal

Asistente conversacional

El asistente conversacional del proyecto PDP-767 fue desarrollado en SRV-PYT-785 y se trata de un modelo/flujo/patrón RAG, Retrieval Augmented Generation, por sus siglas en inglés.

El flujo RAG es una metodología que combina la recuperación de información con la generación de contenido. Este enfoque permite al modelo acceder a conocimientos externos que no forman parte de su entrenamiento inicial, sin necesidad de realizar reentrenamiento. De esta manera, el modelo puede utilizar datos específicos y actualizados. Además, en este proceso, el modelo responde únicamente con la información recuperada de la base de conocimientos, asegurando que las respuestas sean estrictamente basadas en esa información.

Los componentes estructurales de un flujo RAG son los siguientes:

- **El modelo de embeddings y la Base de Conocimientos:**
    - El modelo de embeddings, en este caso el modelo de OpenAI text-embedding-large-3, se encarga de transformar las consultas del usuario y los documentos de la base de conocimientos en vectores numéricos. Estos vectores permiten realizar búsquedas eficientes y precisas en la base de conocimientos, gracias a lo que se conoce como query vectorial, de esta manera se logra identificar la información más relevante en función de la consulta.
    - La base de conocimientos, en este caso Azure AI Search, es un repositorio estructurado o no estructurado que contiene toda la información externa que el sistema puede consultar para obtener datos específicos y actualizados.
- **El modelo Largo de Lenguaje y el prompt:**

  El modelo largo de lenguaje, en este flujo, es gpt-4o-mini, que se utiliza para comprender, contextualizar y generar respuestas coherentes y completas. Este

Internal

componente recibe la información recuperada de la base de conocimientos y la procesa para construir una respuesta adecuada.

Además, para que las respuestas generadas por el modelo, sigan los estándares establecidos por el negocio, se diseña y desarrolla un ***prompt***, este último definirá el estilo, comportamiento e instrucciones de las respuestas generadas.

- **La administración de la conversación:**

  Este componente se encarga de gestionar todo el contexto de la interacción, incluyendo el seguimiento de las preguntas, las respuestas previas y la coherencia del diálogo. Para administrar la conversación el proyecto se apalanca del servicio Azure ComosDB for Mongo.

Para el proyecto PDP-767, el modelo de embeddings (text-embedding-large-3) y el modelo largo de lenguaje (gpt-4o-mini) serán consumidos mediante SecureGPT.

Flujo de consulta

Para el asistente conversacional, los servicios que forman parte del flujo “end-to-end” son los siguientes:

- Azure Cosmos DB for Mongo. Almacenamiento y recuperación de conversaciones.
- Secure GPT. Modelo de embeddings y Modelo Largo de Lenguaje.
- Azure AI Search. Base de conocimienos/Base de datos vectorial.

El flujo de consulta sigue los siguientes pasos:

1. Generar un **conversation_session_id**. Esto se logra ejecutando una petición hacia la ruta “**/conversation/**”. En el backend, este ID representa la creación y guardado efectivo de una conversación en Azure Cosmos DB for Mongo.
2. Usando el ID del punto anterior, ejecutamos una petición hacia la ruta “{conversation_session_id}/message”. Es realmente esta ruta la encargada de ejecutar el flujo RAG, y el flujo de consulta completo es el que sigue:
   I. Se recupera la conversación desde Azure Cosmos DB for Mongo.

Internal

II. Se estructura la conversación junto con la pregunta recién hecha por el usuario para obtener una cadena de texto que representa los últimos diez mensajes que forman parte de la conversación.

III. Se ejecuta la generación del vector embebido asociado a la conversación, para ello, se ejecuta una petición a SecureGPT, hacia la ruta del modelo de embeddings.

IV. Con el vector resultante, se ejecuta una query vectorial hacia AI Search. El resultado de esta query es una lista de extractos de texto asociados a los documentos que ya fueron procesados.

V. Con los resultados de la query vectorial, se recuperan resúmenes y metadatos de los documentos desde Azure Cosmos DB for Mongo.

VI. Con los resúmenes, metadatos y extractos de texto, se genera una cadena de texto que organiza toda la información, es decir, agrupa los extractos de texto, resúmenes y metadatos en función del ID de documento, ordena en función de la fecha de resolución del caso y genera una cadena de texto que conjunta toda esta información. A esta cadena de texto le llamamos **“contexto”.**

VII. Se calcula el indicador de éxito como el conteo de casos que la aseguradora ha ganado en contraste con el total de casos más similares a la pregunta del usuario.

VIII. Al “prompt” que define el comportamiento del asistente, se le añaden el contexto y el indicador de éxito. A esto le llamamos **“prompt aumentado”**

IX. Con el prompt aumentado, los últimos diez mensajes de la conversación y la consulta actual del usuario, se ejecuta la generación de texto hacia el modelo largo de lenguaje de SecureGPT. La respuesta es devuelta en streamming.

X. Una vez que la generación de texto concluyó, entonces el flujo RAG ha concluído. Así que se toma el total de la respuesta generada por el modelo largo de lenguaje, y se le añade a los mensajes de la conversación.

Internal

XI. Finalmente, guardamos la conversación actualizada en Azure Cosmos DB for Mongo.

Contexto

Como se mencionó antes, llamamos “contexto” al resultado del procesamiento de los resultados devueltos por query vectorial, dicho contexto debe ser preciso y debe estar relacionado con el hilo de la conversación entre el usuario y la IA. Por ello, el contexto se obtiene de la siguiente forma:

- La query vectorial devuelve un score de similitud cuyo valor se encuentra en el rango 0 – 1. Mientras más cercano a 1, más similitud.
- La query vectorial recupera los diez extractos con mejor similitud. Sin embargo, estos diez extractos de texto deben cumplir con el requerimiento de tener una similitud superior a un umbral establecido, al momento de la escritura del presente texto, este umbral tiene un valor de 0.7.
- Todos los extractos de texto cuya similitud sea menor al umbral, NO son utilizados para la generación de las respuestas, de esta manera nos aseguramos de que el modelo cuenta con la información más precisa.
- La elección del umbral derivó de pruebas empíricas hechas por el equipo de desarrollo.

Además, el contexto en sí mismo necesita de información adicional para que la calidad de la respuesta sea todavía mayor. Para lograr este cometido se realizan lo siguiente:

- Usando los extractos de texto devueltos por la query vectorial, se recuperan metadatos asociados al (los) documento(s) que dieron origen a los extractos, entre ellos la fecha de resolución y el resumen del caso.
- Se priorizan los casos usando la fecha de resolución al ordenar cada uno de ellos en función de esta.
- Se aumenta la información relevante del documento, es decir, se añade el título, el magistrado, la fecha de resolución el resumen y se ordenan los extractos en función de su aparición en el texto (extracto 1, página 2, etcétera).

Internal



- Se organiza la información usando etiquetas XML, resultado de las buenas prácticas de prompt enginnering.
- Se añade el contexto al prompt.

Con todos estos pasos, se construye un contexto claro y jerárquico que el modelo largo de lenguaje usará para responder a las preguntas del usuario.

Las funciones que ejecutan todo este procesamiento pueden localizarle en el repositorio, en el archivo con la ruta siguiente:

“app\api\core\rag\docsprocessing.py”

Prompt

El prompt es un conjunto de instrucciones, contexto inicial y/o información específica que se proporciona al modelo de lenguaje antes de que genere una respuesta. En un flujo RAG, el prompt juega un papel fundamental, ya que ayuda a guiar al modelo para que produzca respuestas coherentes, relevantes y alineadas con la consulta del usuario y la información recuperada de la base de conocimientos.

El uso del prompt en RAG se centra en estructurar y contextualizar la entrada para que el modelo tenga una guía clara sobre qué tipo de respuesta se espera. Por ejemplo, puede incluir instrucciones específicas y/o fragmentos de información relevante, de esta forma se asegura que el modelo utilice únicamente dicha información y presente la respuesta de manera adecuada.

El prompt del desarrollo SRV-PYT-785 se encuentra en el archivo con la ruta:

“app\api\core\prompts\prompts.py”

Uso de historial

El historial de la conversación permite al asistente mantener el contexto y la coherencia en la interacción con el usuario. Para ello, se utilizan los últimos 10 mensajes intercambiados, que actúan como memoria activa. Esta selección de mensajes recientes ayuda al sistema a entender el tema en curso, responder de manera más natural y evitar

Internal

repeticiones o malentendidos, garantizando una experiencia de diálogo fluida y contextualizada.

Guardado de conversaciones

Para el proyecto PDP-767 en el desarrollo del chatbot conversacional SRV-PYT-785, el código asociado al guardado de conversaciones corresponde con el código de los archivos siguientes:

- ***"app\api\services\azure\azure_comosdb.py"***. En este archivo se encuentra el código asociado a la definición del cliente que se conecta e interactúa con Azure Cosmos DB for Mongo. El método de autenticación es a través del Service Principal (SPN) definido en conjunto con el equipo de arquitectura, así que las variables necesarias para lograr la definición del cliente son:
    - AZURE_COSMOSDB_SUBSCRIPTION_ID
    - AZURE_COSMOSDB_RESOURCES_GROUP_NAME
    - AZURE_CLIENT_ID
    - AZURE_CLIENT_SECRET
    - AZURE_TENANT_ID
- ***"app\api\core\chat\messages.py".*** En este archivo se encuentra el código las funciones que cargan, actualiza y recuperan las conversaciones de los usuarios en Azure Cosmos DB for Mongo.

Guardar las conversaciones en una base de datos, en este caso se en Azure Cosmos DB for Mongo, es fundamental para mantener el contexto y la coherencia en interacciones futuras con el usuario. Esto permite que el sistema tenga acceso a un historial completo o parcial de la interacción previa, lo que mejora la calidad del diálogo.

Campos que se almacenan en la conversación:

- **conversation_session_id:**
  Identifica de manera única la sesión de conversación. Se usa para agrupar todos los mensajes de una misma interacción en diferentes momentos o sesiones.

- **user_id:**
  Identifica al usuario que participa en la conversación, permitiendo personalizar y gestionar sus interacciones de forma individual.
- **conversation_name:**
  Es un nombre descriptivo para la conversación, útil para identificar o categorizar diálogos específicos.
- **start_conversation_timestamp:**
  Marca la fecha y hora en que inició la conversación, útil para análisis temporal y gestión de sesiones.
- **table_counter:**
  Es un contador del número de tablas que se han generado.
- **messages:**
  Es una lista que almacena cada interacción del diálogo, incluyendo tanto las entradas del usuario como las respuestas del asistente.
    - **interaction_id:**
      Identificador único para cada mensaje dentro de la conversación, como "InitialMessage" o un identificador generado por el sistema.
    - **usage_metadata:**
      Información sobre el uso de recursos del modelo, como tokens utilizados y modelos empleados.
    - **role:**
      Indica si el mensaje fue enviado por el usuario o por el asistente, diferenciando los tipos de interacción.
    - **content:**
      El texto real del mensaje, ya sea la instrucción del usuario o la respuesta del asistente.
    - **timestamp:**
      Marca el momento exacto en que se generó cada mensaje, permitiendo reconstruir la secuencia temporal y analizar el flujo de la conversación.

Internal

- **document_ids:**
  En las interacciones del usuario, este campo suele estar vacío, ya que los ID de documentos están relacionados con las respuestas generadas por el modelo RAG. Cuando el asistente genera una respuesta basada en documentos recuperados, estos IDs se almacenan aquí para rastrear qué documentos fueron utilizados en esa interacción. Esto es importante para auditoría, trazabilidad y mejorar los procesos de recuperación y generación.

Indicador de éxito

Para el proyecto PDP-767 en el desarrollo del chatbot conversacional SRV-PYT-785, el código asociado al cálculo del indicador de éxito se localiza en los archivos:

- ***"app\api\core\rag\sucessful_case_statistic.py".*** Archivo que calcula el indicador de éxito.

El asistente es capaz de brindar un panorama de éxito fundamentado en las sentencias más similares al hilo de la conversación que el usuario está llevando actualmente, a este panorama indicador de éxito y se calcula con el siguiente algoritmo:

- Se obtienen los cien extractos más similares a la conversación.
- Se obtienen los metadatos de los documentos de origen asociados a los cien extractos. (NOTA: Recordar que la relación documento – extracto es del tipo uno a muchos).
- Se calcula el número de documentos con estatus de triunfo a favor de la aseguradora y se compara contra el total de documentos de la forma "La aseguradora ha ganado 'n' de los 'N' casos legales previos más similares". Este es el indicador de éxito.
- Se aumenta el prompt del asistente con el indicador de éxito.
- Cuando el usuario ha preguntado explícitamente por la probabilidad de éxito, entonces el asistente usará el indicador de éxito para responder a la pregunta. Además, se generará una tabla con la información de los metadatos asociados a cada uno de los documentos.

Internal

Seguridad y privacidad

Dado que las sentencias judiciales contienen información sensible sobre personas, víctimas, acusados o testigos, así como datos relacionados con delitos y antecedentes, es fundamental manejar su contenido con cuidado. La exposición de esta información puede afectar la privacidad y seguridad de los individuos involucrados.

En PDP-767 IA Sentencias Judiciales, las sentencias judiciales son la fuente principal de información. Para mitigar el impacto de una posible filtración, es necesario eliminar toda la información sensible de estos documentos. Con este objetivo, se desarrolló un servicio que busca eliminar o reducir la información sensible en textos planos. Es crucial asegurarnos de que este servicio funcione adecuadamente, lo que hace que una evaluación precisa sea fundamental en el proceso.

Modelo de Spacy

En esta sección se describen las metodologías diseñadas e implementadas para la evaluación de la primera capa de censura. El resultado obtenido, a grandes rasgos, es una "precisión" de censura del **81.3%.**

La evaluación de este componente nos permite dimensionar el desempeño real de la herramienta. Por lo tanto, proponemos metodologías de etiquetado de datos y de evaluación que satisfagan estas necesidades de manera eficiente. Tras implementar estas metodologías, se obtuvo una "precisión" en el censurado de **81.3%,** lo que indica la efectividad de la censura al evitar la eliminación excesiva o insuficiente de información. Sin embargo, también se consideran otras métricas que pueden ser relevantes.

Internal

Etiquetado de datos

Para el etiquetado de datos, se consideraron 9 documentos que contenían entre 10 y 157 páginas, haciendo un total de 555 páginas. Se extrajo el contenido en texto plano de cada documento y se procesaron las páginas a través de un modelo de lenguaje desplegado localmente. El modelo utilizado para el etiquetado fue Phi-4-mini-instruct , al que se le solicitó extraer información considerada sensible. Esta información se almacenó junto con su texto plano.

Tras revisar el etiquetado hecho por el modelo de lenguaje, notamos que no toda la información extraída era candidata a ser censurada, por ello, de forma manual, realizamos la corrección de los errores cometidos por el modelo.

Una vez corregidos los errores, se aplicó el componente de censura para verificar si había errores adicionales en la revisión humana. Este proceso de revisión y aplicación del modelo se realizó de manera iterativa hasta que no se detectaron omisiones en la censura.

IMPORTANTE:

La aplicación del servicio de censura no introduce sesgo en la evaluación, ya que se trató únicamente de un proceso sin ajustes, funcionando como un "preprocesamiento".

Posteriormente, se llevó a cabo la censura del texto revisado. Para asegurarnos de que no faltara información sensible, se pasó el texto censurado a través de Secure GPT al modelo de lenguaje OpenAI 4o-mini , pidiendo que detectara información sensible. Al igual que en el modelo local, se corrigieron algunos errores de forma manual.

Con este proceso implementado, se generó el conjunto de datos utilizado para ajustar el proceso y su evaluación.

Internal

Separación de datos

Se seleccionaron manualmente los datos utilizados para el entrenamiento, la validación y la prueba, separando documentos completos y considerando únicamente el número de páginas, sin tener en cuenta el contenido, para evitar sesgos. Se buscó mantener una proporción similar entre los conjuntos. Como resultado, se obtuvo aproximadamente un 34% para el entrenamiento, un 38% para la validación y un 28% para la prueba.

Ajuste y evaluación del proceso

El proceso de censura se divide en dos componentes fundamentales para la detección de información a censurar:

1. **Expresiones regulares**: Se utilizan para identificar textos que siguen ciertos patrones, permitiendo detectar las siguientes entidades:

    a. **ID**: Se refiere a cualquier entidad alfanumérica que pueda identificar a una persona, como números de placas, contratos o pólizas.
    b. **EMAIL**: Se refiere a cualquier cadena que parezca un correo electrónico.

2. **Modelo preentrenado de reconocimiento de entidades nombradas (NER)**: Este modelo identifica las siguientes entidades:

    a. **PER**: Nombres de personas.
    b. **ORG**: Nombres de organizaciones.
    c. **LOC**: Localizaciones.
    d. **MISC**: Otras entidades que no se censuran.

Como primer paso, se realizaron múltiples ajustes al proceso de expresiones regulares utilizando los datos de entrenamiento. Este proceso fue el único considerado, ya que el modelo de NER es preentrenado y no se le hicieron ajustes.

Una vez refinado el proceso de censura con expresiones regulares, se llevó a cabo la evaluación en el conjunto de validación. Al obtener métricas satisfactorias, se procedió a evaluar en el conjunto de prueba. Esta evaluación es sin sesgo, ya que no se utilizó ninguna de su información para refinar el proceso.

Las evaluaciones se dividen en dos categorías:

1. **Por palabras**: Esta evaluación considera cada acierto o error a nivel de palabra. Si una entidad está compuesta por más de una palabra, una censura parcial, como censurar un apellido en un nombre, se considera válida para la palabra censurada, pero no para la información no censurada. A continuación, se presenta una tabla ejemplificativa:

Entidad a censurar	Censura del servicio	Errores	Aciertos
Pedro Pérez Ramírez	Pedro **** Ramírez	2	1
Juan Samuel Flores	Juan **** ****	1	2
AXA Seguros México	**** **** ****	0	3

En esta tabla se muestran tres entidades que deben ser censuradas en un documento. En el primer caso, se cuentan 2 errores porque "Pedro" y "Ramírez" no fueron censurados. En el segundo caso, se considera un error debido a que una palabra no fue censurada. Finalmente, en el tercer ejemplo se contabilizan tres aciertos, ya que las tres palabras fueron censuradas correctamente.

2. **Por frases**: A diferencia de la evaluación por palabras, en la evaluación por frases no se consideran correctas las censuras parciales. Si toda la frase es censurada, se considera un acierto; si solo se censura parcialmente o no se censura, se considera un error. A continuación, se presenta una tabla ejemplificativa:

Internal

Entidad a censurar	Censura del servicio	Errores	Aciertos
Pedro Pérez Ramírez	Pedro **** Ramírez	1	0
Juan Samuel Flores	Juan **** ****	1	0
AXA Seguros México	**** **** ****	0	1

En esta tabla se muestran tres entidades que deben ser censuradas en un documento. La primera y la segunda se consideran errores, ya que las entidades no fueron completamente censuradas. En cambio, la tercera se considera un acierto porque la frase fue censurada en su totalidad.

Resultados

1. **Por palabras**: En la evaluación por palabras, el sistema de censura mostró un rendimiento sólido, alcanzando un F1 de **81.3%** en el conjunto de prueba.

IMPORTANTE:
Los valores de entrenamiento y validación son solo informativos y no deben considerarse, ya que se utilizaron para ajustar el proceso. Por lo tanto, estas métricas son propensas a reflejar sobreajuste o sesgos optimistas .

Las métricas mostradas debajo pueden interpretarse como sigue:

- **Precision**: Mide la capacidad del modelo para no censurar información que no es sensible.
- **Recall**: Evalúa la capacidad del modelo para no omitir ninguna palabra en una entidad.

Internal



- **F1**: Indica la precisión del modelo al censurar únicamente la información que debe ser censurada, evitando censurar información adicional o dejar de censurar información sensible.

Las distribuciones (indicadas en la tabla con el prefijo "Dist.") muestran el promedio de cada métrica y su desviación estándar, indicando cuán lejos pueden estar de la media.

En la sección Conclusiones se discutirá brevemente por qué se eligió reportar el F1 en esta evaluación.

Evaluación	**F1**	**Precision**	**Recall**	**Max F1**	**Max Precision**	**Max Recall**	**Min F1**	**Min Precision**	**Min Recall**	**Dist. F1**	**Dist. Precision**	**Dist. Recall**
Entrenamiento	0.879	0.915	0.846	0.970	0.949	0.993	0.708	0.567	0.677	0.845 ± 0.106	0.838 ± 0.157	0.885 ± 0.123
Validación	0.775	0.808	0.745	0.833	0.893	0.780	0.558	0.524	0.596	0.730 ± 0.123	0.754 ± 0.164	0.712 ± 0.082
Test	**0.813**	0.797	0.829	0.833	0.845	1.000	0.557	0.386	0.821	0.695 ± 0.138	0.615 ± 0.229	0.910 ± 0.090

Internal

2. **Por frases:** En la evaluación por frases, el sistema obtuvo un F1 de **66.7%** en el conjunto de prueba. Los resultados pueden interpretarse de manera similar a la sección anterior.

Es importante destacar que la evaluación por frases es mucho más estricta que la evaluación por palabras, lo que explica que las métricas sean menos favorables. Más adelante se discutirán las diferencias entre ambas evaluaciones.

Evaluación	**F1**	**Precision**	**Recall**	**Max F1**	**Max Precision**	**Max Recall**	**Min F1**	**Min Precision**	**Min Recall**	**Dist. F1**	**Dist. Precision**	**Dist. Recall**
Entrenamiento	0.836	0.824	0.848	0.879	0.878	0.969	0.444	0.308	0.735	0.732 ± 0.176	0.676 ± 0.220	0.846 ± 0.088
Validación	0.603	0.573	0.636	0.687	0.785	0.760	0.361	0.288	0.484	0.569 ± 0.147	0.566 ± 0.207	0.603 ± 0.116
Test	**0.677**	0.613	0.757	0.713	0.683	1.000	0.370	0.227	0.745	0.542 ± 0.172	0.455 ± 0.228	0.872 ± 0.128

Internal

Conclusiones

Tras revisar los resultados, se considera que el sistema de censura ha mostrado un desempeño sólido, alcanzando un F1 de **0.813** en la evaluación por palabras.

La evaluación por palabras es relevante porque una censura parcial disminuye el número de personas, compañías o entidades que podrían ser identificadas, lo cual cumple con el objetivo del componente. Por ejemplo, si existen 10 clientes de AXA llamados "Pedro Pérez" y otro apellido, es probable que la cantidad de personas llamadas "Pedro Pérez Ramírez" sea menor o igual a 10, donde 10 sería el caso en que todos los "Pedro Pérez" son "Pedro Pérez Ramírez", lo cual es poco probable.

Además, es importante destacar que se reporta una métrica que representa un balance entre no censurar de más y no censurar de menos. Sin embargo, dado que la prioridad es evitar la censura insuficiente, se debe prestar atención al recall, que en la evaluación por palabras es de **0.829**.

Modelo Grande de Lenguage (LLM)

En esta sección se describen las metodologías diseñadas e implementadas para evaluar la segunda capa de censura. Los resultados obtenidos se fundamentan en métricas basadas en palabras y frases, donde en ambos casos se considerará como métrica principal el ***recall***. Esta elección se justifica porque la relevancia de la censura radica en la protección de información legal y/o sensible, siendo más grave omitir la censura de una palabra crítica (falso negativo) que censurar erróneamente una palabra (falso positivo). Además, se asegura que el sentido del texto no se vea comprometido por una censura excesiva.

La evaluación de este componente nos permite medir el desempeño real de la herramienta. Por lo tanto, proponemos metodologías de etiquetado de datos y de evaluación que satisfagan estas necesidades de manera eficiente. La primera capa de censura se implementa mediante un modelo preentrenado de Reconocimiento de Entidades Nombradas (NER). La información que no puede ser detectada por este modelo se censura a través de un Modelo Largo de Lenguaje (LLM), donde claramente la cantidad

Internal

de información etiquetada como sensible que se le proporciona a este modelo es reducida y ya no permite la identificación de algún dato relacionado a una persona física o moral debido a la aplicación de la primera capa.

Etiquetado de datos

Para el etiquetado de datos, se consideraron 9 documentos que contenían entre 10 y 157 páginas, haciendo un total de 555 páginas. Se utilizó el modelo Phi-4-mini-instruct para identificar información sensible, la cual se almacenó junto con el texto plano. Al revisar el etiquetado, se detectó que no toda la información extraída era susceptible de censura, por lo que se realizaron correcciones manuales a los errores del modelo. Este proceso se realizó de manera iterativa, aplicando el componente de censura para verificar posibles omisiones. Es importante destacar que la aplicación del servicio de censura no introdujo sesgo, ya que se trató de un proceso sin ajustes. Luego, el texto censurado fue revisado con Secure GPT y el modelo de OpenAI 4o-mini para detectar información sensible, corrigiendo manualmente nuevos errores. Finalmente, se generó un conjunto de datos para ajustar y evaluar el proceso.

IMPORTANTE:
La aplicación del servicio de censura no introduce sesgo en la evaluación, ya que se trató únicamente de un proceso sin ajustes, funcionando como un "preprocesamiento".

Una vez obtenidas las entidades de este primer proceso, se obtuvo una etiqueta asociada a cada entidad que define el tipo de dato y la indicación de si debe ser censurada o no, además del texto plano censurado. Este nuevo texto censurado se envía al Modelo Largo de Lenguaje, que ayuda a detectar aquellas entidades que no lograron censurarse en la primera capa.

IMPORTANTE:
La información enviada a SecureGPT ya no es considerada sensible. Aunque puede haber algún nombre o dato que la primera capa no haya detectado, ya no se trata de nombres completos o información que pueda asociarse a una persona específica.

Internal

Con este segundo proceso implementado, se generó el conjunto de datos utilizado para ajustar el proceso y su evaluación.

Separación de datos

Se seleccionaron manualmente los datos utilizados para el entrenamiento, la validación y la prueba, separando documentos completos y considerando únicamente el número de páginas, sin tener en cuenta el contenido, para evitar sesgos. Se buscó mantener una proporción similar entre los conjuntos. Como resultado, se obtuvo aproximadamente un 34% para el entrenamiento, un 38% para la validación y un 28% para la prueba.

Ajuste y evaluación del proceso

El proceso de la primera capa de censura se divide en dos componentes fundamentales para la detección de información a censurar:

1. **Expresiones regulares**: Se utilizan para identificar textos que siguen ciertos patrones, permitiendo detectar las siguientes entidades:
    a. **ID**: Se refiere a cualquier entidad alfanumérica que pueda identificar a una persona, como números de placas, contratos o pólizas.
    b. **EMAIL**: Se refiere a cualquier cadena que parezca un correo electrónico.
2. **Modelo preentrenado de reconocimiento de entidades nombradas (NER)**: Este modelo identifica las siguientes entidades:
    a. **PER**: Nombres de personas.
    b. **ORG**: Nombres de organizaciones.
    c. **LOC**: Localizaciones.
    d. **MISC**: Otras entidades que no se censuran.

Como primer paso, se realizaron múltiples ajustes al proceso de expresiones regulares utilizando los datos de entrenamiento. Este proceso fue el único considerado, ya que el modelo de NER es preentrenado y no se le hicieron ajustes.

Internal

Una vez refinado el proceso de censura con expresiones regulares, se llevó a cabo la evaluación en el conjunto de validación. Al obtener métricas satisfactorias, se procedió a evaluar en el conjunto de prueba. Esta evaluación es sin sesgo, ya que no se utilizó ninguna de su información para refinar el proceso.

Posteriormente en la segunda capa, al pasar al LLM el texto censurado obtenido de la primera capa, también se identifican las siguientes entidades a censurar:

- **PER**: Nombres de personas.
- **POLICY**: Identificadores de números de póliza y contratos.
- **ID**: Números IMEI, RFC y BIN (o números de tarjeta bancaria).
- **ADDRESS**: Direcciones de domicilios particulares.
- **LICENSE_PLATE**: Placas de automóviles o motocicletas.

Las evaluaciones se dividen en dos categorías:

1. **Por palabras**: Esta evaluación considera cada acierto o error a nivel de palabra. Si una entidad está compuesta por más de una palabra, una censura parcial, como censurar un apellido en un nombre, se considera válida para la palabra censurada, pero no para la información no censurada. A continuación, se presenta una tabla ejemplificativa:

Entidad a censurar	Censura del servicio	Errores	Aciertos
Pedro Pérez Ramírez	Pedro **** Ramírez	2	1
Juan Samuel Flores	Juan **** ****	1	2
AXA Seguros México	**** **** ****	0	3

En esta tabla se muestran tres entidades que deben ser censuradas en un documento. En el primer caso, se cuentan 2 errores porque "Pedro" y "Ramírez" no fueron censurados. En el segundo caso, se considera un error debido a que una palabra no fue censurada. Finalmente, en el tercer ejemplo se contabilizan tres aciertos, ya que las tres palabras fueron censuradas correctamente.

Internal

2. **Por frases**: A diferencia de la evaluación por palabras, en la evaluación por frases no se consideran correctas las censuras parciales. Si toda la frase es censurada, se considera un acierto; si solo se censura parcialmente o no se censura, se considera un error. A continuación, se presenta una tabla ejemplificativa:

Entidad a censurar	Censura del servicio	Errores	Aciertos
Pedro Pérez Ramírez	Pedro **** Ramírez	1	0
Juan Samuel Flores	Juan **** ****	1	0
AXA Seguros México	**** **** ****	0	1

En esta tabla se muestran tres entidades que deben ser censuradas en un documento. La primera y la segunda se consideran errores, ya que las entidades no fueron completamente censuradas. En cambio, la tercera se considera un acierto porque la frase fue censurada en su totalidad.

Resultados

1. **Por palabras:** En la evaluación por palabras, el sistema de segunda censura mostró un rendimiento sólido o uniforme en el **recall** que oscila sobre el **90%** en los tres conjuntos de información evaluada, lo que indica que esta métrica no presenta sobreajuste pues sus valores son similares, es decir cuando se evalua un nuevo conjunto de información, el modelo no memoriza los datos de entrenamiento y en realidad sí aprende patrones que se repiten en datos nuevos, lo cual indica una buena generalización.

Las métricas mostradas debajo pueden interpretarse como sigue:

- **Precision**: Mide la capacidad del modelo para no censurar información que no es sensible.

Internal

- **Recall**: Evalúa la capacidad del modelo para no omitir ninguna palabra que debe ser censurada
- **F1**: Indica la precisión del modelo al censurar únicamente la información que debe ser censurada, evitando censurar información adicional o dejar de censurar información sensible.

Las distribuciones (indicadas en la tabla con el prefijo "Dist.") muestran el promedio de cada métrica y su desviación estándar, indicando cuán lejos pueden estar de la media.

Evaluación	**F1**	**Precision**	**Recall**	**Max F1**	**Max Precision**	**Max Recall**	**Min F1**	**Min Precision**	**Min Recall**	**Dist. F1**	**Dist. Precision**	**Dist. Recall**
Entrenamiento	0.708	0.572	0.929	0.878	0.803	0.994	0.165	0.09	0.907	0.651 ± 0.29	0.561 ± 0.288	0.952 ± 0.032
Validación	0.703	0.581	0.889	0.736	0.627	0.891	0.646	0.507	0.88	0.684 ± 0.038	0.559 ± 0.05	0.887 ± 0.005
Prueba	0.335	0.205	**0.903**	0.777	0.684	1.000	0.002	0.205	0.9	0.398 ± 0.387	0.347 ± 0.337	0.95 ± 0.05

Internal

2. **Por frases:** En la evaluación por frases, el sistema de segunda censura obtuvo un Recall de **93.6%** en el conjunto de prueba. Los resultados pueden interpretarse de manera similar a la sección anterior.

Es importante destacar que la evaluación por frases es mucho más estricta que la evaluación por palabras, lo que explica que algunas métricas sean menos favorables. Más adelante se discutirán las diferencias entre ambas evaluaciones.

Evaluación	**F1**	**Precision**	**Recall**	**MaxF1**	**Max Precision**	**Max Recall**	**MinF1**	**Min Precision**	**Min Recall**	**Dist. F1**	**Dist. Precision**	**Dist. Recall**
Entrenamiento	0.638	0.495	0.897	0.786	0.687	0.949	0.182	0.101	0.872	0.56 ± 0.227	0.442 ± 0.213	0.916 ± 0.028
Validación	0.652	0.528	0.851	0.665	0.564	0.892	0.642	0.517	0.803	0.657 ± 0.01	0.537 ± 0.02	0.847 ± 0.036
Test	0.274	0.161	**0.936**	0.789	0.683	1.000	0.01	0.005	0.935	0.4 ± 0.39	0.161 ± 0.339	0.968 ± 0.032

Internal

Conclusiones

Tras revisar los resultados, se concluye que el sistema de censura ha demostrado un desempeño sólido, alcanzando un Recall de **90.3%** en la evaluación por palabras.

La evaluación por palabras es relevante porque una censura parcial disminuye el número de personas, compañías o entidades que podrían ser identificadas, lo cual cumple con el objetivo del componente. Por ejemplo, si existen 10 clientes de AXA llamados "Pedro Pérez" y otro apellido, es probable que la cantidad de personas llamadas "Pedro Pérez Ramírez" sea menor o igual a 10, donde 10 sería el caso en que todos los "Pedro Pérez" son "Pedro Pérez Ramírez", lo cual es poco probable.

Es importante destacar que la métrica evaluada en esta capa de censura es el Recall, ya que la prioridad es evitar la censura insuficiente. Aunque la Precisión ha disminuido considerablemente, esto se debe a que ahora se censuran más palabras de las que realmente se consideran necesarias, es decir, se identifican como sensibles palabras que no deberían ser censuradas. Esto podría parecer contraproducente, ya que podría generar la preocupación de que se pierda el sentido del texto. Sin embargo, al analizar este fenómeno, se observa que el LLM puede censurar palabras como "paciente", "familia", "hijo", "demandado", "actor", "Sr.", entre otras, e incluso marcas como *Ford* se pueden clasificar erróneamente como entidades tipo persona. Esto, sin duda, incrementa el número de entidades que no deben ser censuradas, pero claramente no afecta el sentido del texto.

Por ejemplo, en el siguiente texto (tomado de una sentencia real, pero con nombres propios reemplazados por aleatorios):

En el proceso: la actora Maria Irene Saldivar Ruíz, el demandado John Williams Swan y la tercera llamada a juicio Romina Leticia Saldivar Ruíz se legitimaron en términos del artículo 55 (fracción I) del Código de Procedimientos Civiles para el Estado de Sonora
La segunda capa de censura puede devolver un resultado como:

En el proceso: la [PERSON] [PERSON] [PERSON] [PERSON] [PERSON], el [PERSON] [PERSON] [PERSON] [PERSON] y la tercera llamada a juicio [PERSON] [PERSON] [PERSON]

Internal

[PERSON] se legitimaron en términos del artículo 55 (fracción I) del Código de Procedimientos Civiles para el Estado de Sonora

Lo cual ejemplifica que la censura de las palabras *actora* y *demandado* no altera el sentido del texto.

Internal

Evaluación y métricas

Además de las métricas que permiten evaluar la calidad de la despersonalización, se cuenta con algunas otras que ayudan a evaluar el rendimiento del proyecto.

Tiempos de procesamiento

Las siguientes tablas se presenta únicamente como referencia, con el propósito de ilustrar los tiempos estimados de procesamiento observados en el ambiente de QA:

- Para un documento de 13 páginas considerado como mixto, es decir que tiene página nativas y no nativas:

Proceso	Tiempo en mins.
Extracción por OCR	4.646
Primera capa de despersonalización	1.4045
Segunda capa de despersonalización	0.0024
Extracción de entidades relevantes con secureGPT	4.5
Estandarización y limpieza de entidades relevantes	0.000046
Generación de vectores	0.00047
Generación de resumen con secureGPT	0.139

En cuanto a la primera y segunda capa de despersonalización se tienen por página los siguientes tiempos en segundos:

	Mínimo	Máximo	Promedio
Primera capa	0.0052	0.0416	0.011
Segunda capa	4.17	9.62	6.48

Lo que da un resultado de **10:41 minutos** como tiempo total de procesamiento de dicho documento

- Para un documento de 42 páginas considerado nativo:

Internal

Proceso	Tiempo en mins.
Extracción por OCR	7.5636
Primera capa de despersonalización	0.0123
Segunda capa de despersonalización	6.4820
Extracción de entidades relevantes con secureGPT	7.5636
Estandarización y limpieza de entidades relevantes	0.000143
Generación de vectores	0.00077
Generación de resumen con secureGPT	0.4435

En cuanto a la primera y segunda capa de despersonalización se tienen por página los siguientes tiempos en segundos:

	Mínimo	Máximo	Promedio
Primera capa	0.006	0.3613	0.01763
Segunda capa	4.8821	16.0867	9.25

Lo que da un resultado de **22:35 minutos** como tiempo total de procesamiento de dicho documento.

Embeddings

En esta sección se muestran las características y criterios que fueron usados en la evaluación de modelos de embedidos para el proyecto PDP-767: IA Sentencias Judiciales. Este proyectotiene como objetivo automatizar la búsqueda de sentencias judiciales históricas vinculadas a los litigios de siniestros de AXA Seguros México, así que una parte crucial es la interfaz conversacional que permite interactuar con un modelo de IA. Dicho modelo de IA está fundamentado en lo que se conoce como patrón RAG, Retrieval Augemented Generation por sus siglas en inglés.

Internal

Patron RAG

El Patrón RAG, Retrieval Augemented Generation por sus siglas en inglés, es un enfoque en inteligencia artificial que combina la búsqueda de información con la generación de texto. En pocas palabras se siguen los siguientes pasos: primero, el sistema busca información relevante sobre el tema que le preguntas. Luego, utiliza esa información para crear una respuesta coherente y útil.

En el presente texto se hablará sobre los criterios de selección del modelo de embedidos, el componente principal de la búsqueda de información en el patrón RAG.

Para obtener más información sobre el patrón RAG, haz clic aquí

AXA Secure GPT Model Hub

Secure GPT Model Hub es un producto interno, creado por AXA, para permitir a todos los miembros de la empresa llevar a cabo casos de uso de IA Generativa con una API segura y confiable.

Su objetivo es ofrecer a los desarrolladores un acceso seguro a los modelos de IA generativa más grandes y genéricos, de forma que puedan centrarse en crear valor para sus proyectos, en lugar de mantener una plataforma.

Para conocer más sobre Secure GPT, visita Secure GPT Model Hub

- **API**

La API de Scure GPT fue diseñada para poder conocer los modelos disponibles, sus capacidades, sus proveedores asociados y la forma de interactuar con estos.

Para el presente texto, la API resulta sumamente relevante, pues a través de ella se consultaron todos los modelos de embedidos disponibles.

Para obtener más información sobre la API, visita Secure GPT Model Hub API

- **Modelo de Embeddings**

Internal



Los modelos de embebidos son modelos que permiten transformar texto en vectores numéricos (un vector numérico es realmente una lista de números). Esta representación numérica-vectorial facilita la comparación entre textos.

En el contexto del Patrón RAG, los modelos de embebidos son un componente importante, pues se utiliza la representación numérica-vectorial de las consultas del usuario para compararse contra las representaciones numérico-vectorial de textos que conforman lo que se conoce como base de conocimientos.

Los modelos de embedidos disponibles en Secure GPT Model Hub pueden ser consultados a través de la ruta de la API siguiente:

/providers

Al ejecutar una solicitud a la ruta antes mencionada, la respuesta recibida contendrá el conjunto de todos los proveedores disponibles en Secure GPT Model Hub así como el conjunto de todos los modelos disponibles asociados a cada proveedor.

Para el presente texto se ejecutó el ejercicio de consulta y al momento de la escritura de este (08/04/2025) se concluyó que los modelos de embedidos disponibles en Secure GPT Model Hub son lo siguientes:

- Open AI: text-embedding-ada-002
- Open AI: text-embedding-3-large

La misma conclusión puede ser deducida al visitar y revisar el apartado de modelos de Secure GPT Model Hub .

Metodología de evaluación

Para evaluar los modelos de embedidos disponibles en Secure GPT Model Hub, se siguió la siguiente metodología:

Internal



1. Investigación sobre el desempeño de los modelos de embedidos disponibles en Secure GPT Model Hub.
2. Análisis del desempeño de los modelos de embedidos disponibles en Secure GPT Model Hub.
3. Elección del mejor modelo disponible en Secure GPT Model Hub basado en su desempeño

Investigación sobre el desempeño de los modelos de embedidos

Derivado de los ejercicios de investigación realizados por el equipo de desarrolladores, a continuación, se resumen los hallazgos encontrados:

- El desempeño de los modelos ha sido medido con MTEB (Massive Text Embeddings Benchmark).
- La métrica de evaluación usada para calificar a los modelos de embedidos es la nDGC (Normalized Discounted Cumulative Gain).
- El modelo "text-embedding-ada-002" se ve superado por el modelo "text-embedding-3-large".

MTEB Massive Text Embeddings Benchmark

MTEB es un proyecto de evaluación de modelos de embedidos que engloba una cantidad considerable de tareas. A su vez, MTEB nos permite visualizar un tablero de liderazgo que lista los modelos de embedidos y sus métricas de evaluación asociadas.

Para conocer más sobre METB, visita MTEB

Para visualizar el tablero de liderazgo de MTEB, visita MTEB Leaderboard

Para profundizar en la metodología técnica de MTEB, refiere a arXiv y para profundizar en la forma de evaluación para el tipo de tarea "Retrieval" refiere a arXiv .

Internal

nDCG

La métrica nDCG (Normalized Discounted Cumulative Gain) es una forma de evaluar la calidad de los resultados de búsqueda en sistemas de recuperación/consulta de información. Se centra en dos aspectos clave: la relevancia de los resultados y su posición en la lista devuelta por el sistema de recuperación/consulta. La nDCG mide cuán bien un sistema presenta respuestas relevantes, penalizando las respuestas a través de sus posiciones en la lista, es decir, respuestas en las posiciones más bajas son mayor penalizadas que respuestas en las posiciones más altas. La puntuación se normaliza para estar entre 0 y 1, donde 1 representa el mejor desempeño posible.

Dada la naturaleza de la métrica, su uso para evaluar el componente de búsqueda de información en los patrones RAG ha sido adoptado por MTEB.

Para profundizar en la metodología técnica de nDGC, refiere a arXiv .

Análisis del desempeño de los modelos de embedidos.

Interpretación del tablero de liderazgo MTEB

Como ya se ha mencionado, MTEB es una comparación multitarea de modelos de embedidos. Se presenta en forma de tablero de liderazgo en el que la posición del modelo está basada en múltiples puntuaciones, de hecho, estas múltiples puntuaciones se dividen en los 8 grandes tipos de tareas que enlista MTEB:

- Bitext Mining.
- Classification.
- Clustering.
- Pair Classification.
- Reranking.
- Retrieval.

Internal



- Semantic Textual Similarity (STS).
- Summarization

De la anterior lista de tipos de tareas es completamente necesario hacer énfasis en el tipo "Retrieval", pues es este tipo de tarea el que evalúa el desempeño del modelo de embedidos en tareas de recuperación/consulta de información.

Ahora, cada modelo presente en el tablero de liderazgo de MTEB puede tener una puntuación asociada a cada uno de los tipos de tarea presentados arriba, es importante aclarar que esta puntuación es resultado de la agregación de puntuaciones individuales en tareas que cumplen con los criterios para ser categorizadas en alguno de estos tipos, en otras palabras, el puntaje de desempeño de un modelo de embedidos 'A' en el tipo de tarea 'Retrieval' es el agregado de todos los puntajes individuales del mismo modelo 'A' en cada una de las tareas que son del tipo 'Retrieval', de hecho, cada tarea puede pensarse como un conjunto de datos individual en el que podemos evaluar nuestro modelo, como existen múltiples conjuntos de datos entonces el resultado final se logra agregando estos resultados individuales.

Debido a la diversidad de tareas/conjuntos de datos se debe procurar no limitarse a mirar las puntuaciones de los modelos en el tipo de tarea de interés. En su lugar, se debe procurar analizar las puntuaciones individuales de las tareas/conjuntos de datos que mejor representen el caso de uso. Gracias a los estándares de MTEB, las tareas/conjuntos de datos han sido agrupadas en función de características que resultan sumamente útiles, estas características son las siguientes:

- **task_types:** Es el tipo de tarea (arriba se listaron los 8 tipos existentes)
- **domains:** Corresponde con el dominio de conocimiento del texto. Ejemplo: dominio Legal.
- **languages:** Corresponde con el lenguaje en el que se encuentra el texto.
- Así, el caso de uso del proyecto PDP-767: IA Sentencias Judiciales puede verse representado por las siguientes características:

Internal

- **task_types = Retrieval** -> Por tratarse de un patrón RAG.
- **domains = Legal** -> Por tratarse de sentencias judiciales.
- **languages = Español** -> Porque los textos que alimentarán al proyecto están escritos en español.

IMPORTANTE:

Lo mejor es revisar los puntajes de desempeño individuales de los modelos de embedidos disponibles en Secure GPT obtenidos en las tares/conjuntos de datos que empatan con las caracteristicas del caso de uso.

Elección del mejor modelo basado en su desempeño

Finalmente, los resultados MTEB de los modelos disponibles en SecureGPT son los siguientes:

	ada v2	text-embedding-3-small		text-embedding-3-large		
Embedding size	1536	512	1536	256	1024	3072
Average MTEB score	61.0	61.6	62.3	62.0	64.1	64.6

Resultados

La elección del mejor modelo de embedidos ha sido direccionada por los siguientes hechos:

- Los modelos de embedidos de OpenAI son agnosticos al lenguaje pues el texto se procesa hacia UTF-8. Para más información visita el enlace

Internal



- Las métricas MTEB obtenidas por los modelos brindan un esquema de comparación, en el que el modelo con la métrica más alta es aquel que se desempeña mejor.
- En el caso del modelo text-embedding-3-large, la dimensión del embedido resultante puede ser seleccionada de entre los siguientes valores: [256, 1024, 3072]. La relación entre el desempeño del modelo y su dimensión es lineal, es decir, a mayor dimensión mejor desempeño, sin embargo, se debe considerar que la diferencia en el desempeño entre la dimensión 1024 y 3072 es poca en relación con el tamaño del embedido.

Así, el modelo seleccionado para el proyecto será:

- modelo: text-embedding-3-large
- dimensión: 1024
- desempeño MTEB: 64.1

Logs de procesos exitosos y fallidos

La visibilidad de los loggers dependerá de la variable **LOGGING_LEVEL** definida en el archivo .env de cada servicio.Esta variable puede tomar los valores:

DEBUG, INFO, WARN, ERROR y FATAL

Con ese orden de importancia, es decir DEBUG < INFO, etc

De tal forma que cada valor deja mostrar todos los loggers que se hayan definido iguales o por encima de él. Por ejemplo, si en el código se han definido los mensajes:

- logger.info(“Este en un mensaje de información”)
- logger.debug(“Este en un mensaje para debuggeo”)

Y la variable **LOGGING_LEVEL** = DEBUG entonces ambos mensajes serán mostrados, en cambio si el valor es INFO, solo el mensaje del tipo logger.info() se mostrará.

Internal



Internal

BI-040-253 Indicadores de éxito para sentencias en curso

Sobre el origen de datos

El proyecto PDP 767 IA SENTENCIAS JUDICIALES, es un proyecto de extracción de datos a partir de documentos PDF que representan sentencias judiciales atendidas por AXA. Los datos que la solución extrae de cada uno de los PDF procesados son depositados en una instancia de Azure Cosmos DB from Mongo, la información del recurso se lista a continuación:

- Directorio Azure: AXA -> axa365.onmicrosoft.com
- Grupo de recursos: z-nsm-iasentenciasjud-pr01-ue2-01
- Nombre del recurso: iasentenciasjudpr-account
- Nombre de Base de datos: legal-assistant
- Nombre de la colección: document_metadata

A continuación, se presenta una tabla de resumen con los campos que componen a la colección en CosmosDB:

Internal

Campo	Descripción	Ejemplo de Valores
_id	Es un ID único generado por el propio CosmosDB.	ObjectId("688bcd3035afa533049b5380")
hash	Es el UUID identificador del documento que fue procesado.	b8126b27ef1f8e6730a2e68eb91c2b90f5a4cea3c209e420c9e965045beeed27
user_id	Es el ID del usuario quien proceso el documento en la aplicación.	default@domain.com
automatic_validation	Es un valor booleano. TRUE, cuando la validación de la censura del documento fue automática. False en caso contrario.	True
filename	Es el nombre original del archivo que fue procesado.	SENTENCIA-060606.pdf
extension	Es la extensión original del archivo que fue procesado.	pdf
mime_type	Es la definición del tipo de archivo que fue procesado.	application/pdf
size	Es el tamaño del archivo en bytes.	924707
file_id	Es el UUID identificador del documento que fue procesado.	b8126b27ef1f8e6730a2e68eb91c2b90f5a4cea3c209e420c9e965045beeed27
censored_file_id	Es el ID identificador del documento censurado asociado al documento procesado.	censored_b8126b27ef1f8e6730a2e68eb91c2b90f5a4cea3c209e420c9e965045beeed27
upload_timestamp	Es la estampilla de tiempo asociada al momento en el que se proceso el documento en la aplicación. Formato ISO 8601 y zona horaria respecto a UTC-6.	2025-07-31T14:07:43.363424-06:00
doc_type	Es el tipo de documento. Native, cuando el documento es un PDF sin alteraciones. Non-native, cuando el documento es un PDF generado por el escaneo digital de un documento físico.	Native
sentence_resolution_data.presiding_magistrate.name	Es el nombre de pila del magistrado que atendió el caso presentando en el documento procesado en la aplicación.	Luis
sentence_resolution_data.presiding_magistrate.last_name1	Es el primer apellido del magistrado que atendió el caso presentando en el documento procesado en la aplicación.	Fernandez
sentence_resolution_data.presiding_magistrate.last_name2	Es el segundo apellido del magistrado que atendió el caso presentando en el documento procesado en la aplicación.	del Toro
sentence_resolution_data.judicial_circuit	Es el circuito judicial en el que fue atendido el caso presentado en el documento procesado por la aplicación.	1
sentence_resolution_data.state_or_locality	Es el estado o localidad en el que fue atendido el caso presentado en el documento procesado por la aplicación.	ciudad de mexico
sentence_resolution_data.court_number.number	Es el estado o localidad de la corte en el que fue atendido el caso presentado en el documento procesado por la aplicación.	Cuarto de Distrito
sentence_resolution_data.court_number.civil_matters	Es la corte en el que fue atendido el caso presentado en el documento procesado por la aplicación.	Mercantil Especializado en Juicios de Cuantia Menor
sentence_resolution_data.ruling_date	Es la fecha de resolución del caso presentado en el documento procesado por la aplicación. Formato: AAAA-MM-DD	2019-01-30
sentence_resolution_data.decision_outcome	Es un valor booleano. TRUE, cuando AXA ha perdido el caso presentado en el documento, False en caso contrario.	True

Sobre el tablero

Objetivo

Visualizar la información relevante a Autoridad, Fecha de emisión, Materia, Localidad del histórico de sentencias judiciales cargado en la aplicación PDP 767 IA SENTENCIAS JUDICIALES.

Datos que alimentan el tablero

Para el tablero en Power BI, serán usado únicamente los siguientes datos:

- sentence_resolution_data.judicial_circuit

Internal

- sentence_resolution_data.state_or_locality
- sentence_resolution_data.court_number.number
- sentence_resolution_data.court_number.civil_matters
- sentence_resolution_data.ruling_date
- sentence_resolution_data.decision_outcome
- file_id
- upload_timestamp

IMPORTANTE:

El tablero consumirá dichos datos desde una tabla que cumple el siguiente formato y que ya toma en cuenta las transformaciones necesarias para el consumo directo de PowerBI, únicamente se muestra un registro para dar a conocer la estructura:

	FILE_ID	CIRCUIT	LOCALITY	COURT_NUMBER_STR	COURT_MATTERS	COURT_MATTERS_STR	AUTORITY	YEAR	MONTH	DAY	DATE	LOST_CASE
0	8ad002fd7092595b4164b5e31e5c824c689c6bf4f193a0...	1	CIUDAD DE MEXICO	JUZGADO TRIGESIMO CUARTO	CIVIL	CIVIL	JUZGADO TRIGESIMO CUARTO EN MATERIA CIVIL	2021	6	3	2021-06-03	True

Para llegar a dicha tabla se hicieron las siguientes transformaciones:

1. Estandarización de todos los valores tipo None, Nan u otro a la cadena “no detectado”, esto ayudará a no perder de vista aquellas sentencias que no tengan la información completa
2. Cambio de nombres de columnas:
    a. file_id -> FILE_ID
    b. judicial_circuit -> CIRCUIT
    c. state_or_locality -> LOCALITY
    d. court_number -> COURT_NUMBER_STR
    e. civil_matters -> COURT_MATTERS_STR

Internal



f. ruling_date -> DATE

g. decision_outcome -> LOST_CASE

h. upload_timestamp -> UPLOAD_TIMESTAMP

3. Limpiar espacios al inicio y al final de cada registro
4. Quitar acentos, caracteres especiales y conversión a mayúsculas de todos los campos, a excepción de FILE_ID, DATE y UPLOAD_TIMESTAMP
5. Obtención de la materia (COURT_MATTERS) que se obtiene de COURT_MATTERS_STR que es la cadena de texto que el modelo largo de lenguaje asocia a la materia, es decir el texto completo, mientras COURT_MATTERS es solo la materia. Por ejemplo, si COURT_MATTERS_STR es “juicio oral mercantil”, entonces COURT_MATTERS será “mercantil”
6. Obtención de la autoridad (AUTORITY) que se compone del número de tribunal y la materia
7. Parseo de fechas:
    a. Cambio a tipo *datetime*
    b. Creación de variables DAY, MONTH y YEAR
8. Verificación de LOST_CASE que toma 3 valores: true, false y NO DETECTADO

Elementos del tablero

Filtros

- Circuito o Localidad.
    - Tipo de elemento: Lista desplegable.
    - Descripción: Ayuda a filtrar el contenido del tablero en función del circuito o localidad.

Internal

- Fecha de emisión.
    - Tipo de elemento: Línea de tiempo por rangos.
    - Descripción: Ayuda a filtrar el contenido del tablero en función de un rango de fechas establecido.
- Materia.
    - Tipo de elemento: Lista seleccionable.
    - Descripción: Ayuda a filtrar el contenido del tablero al seleccionar una o más materias.
- ¿Caso perdido?
    - Tipo de elemento: Lista seleccionable.
    - Descripción: Ayuda a filtrar el contenido del tablero en función de si los casos fueron Perdidos, es decir, AXA perdió el caso, (True) o no lo fueron (False).
- Autoridad.
    - Tipo de elemento: Lista desplegable.
    - Descripción: Ayuda a filtrar el contenido del tablero en función de la autoridad que atendió el caso.

Medidas

- *Total de casos.* Representa el conteo de casos.
- *Tasa de éxito.* Representa el porcentaje de casos que han sido ganados por AXA.

Visualizaciones

- *Mapa geográfico.* Utiliza un mapa de la República Mexicana y dibuja círculos centrados en el estado, el tamaño del circulo representa el conteo de casos atendidos en dicho estado.

Internal

- *Gráfico de barras apiladas por fecha (Total de casos Ganados y Perdidos).* Pinta una línea de tiempo que permite visualizar una barra por cada día, dicha barra pinta el total de casos como la altura total de cada barra, además, en color rojo oscuro se pintan los casos que AXA ha perdido, en color azul se pintan los casos que AX ha ganado.
- *Gráfico de dona (Casos por materia).* Pinta el total de casos segmentando en función de la materia.
- *Medidor (Tasa de éxito).* Pinta el porcentaje de casos ganados respecto al total de casos.
- *Gráfico de barras por Circuito/Localidad (Porcentaje de casos ganados por circuito o localidad).* Pinta el porcentaje de casos ganados en cada una de las barras a lo largo de los circuitos/localidades.

Se ha validado con el equipo de negocio y se ha desarrollado un tablero con datos simulados, esto con el fin de ejemplificar el resultado final:



Internal

Atención de incidencias y Rollback

Monitoreo en Openshift

En caso de que se busque tener una mayor visibilidad de los procesos ocurridos dentro de los servidores de la aplicación, es posible hacer una solicitud al equipo de CICD en Openshift para la consulta de métricas del sistema (como uso de Memoria RAM, CPU, etc.) y los registros (logs) de de uno o más contenedores. En caso de tener accesos mínimos de visualización, como Viewer, para los ambientes de Openshift, podrá consultar esta información directamente desde la consola en la pestaña de Métricas y la pestaña de Registros de cada Pod.

Openshift también indica un contador de reinicios para cada Pod, este indicador nos ayuda a revisar a hora de reinicio, y el último estatus del Pod antes de ser reiniciado. Estos reinicios del servicio suelen pasar por una sobresaturación de los recursos de memoria (OOM), si tal ocasión ocurre, es recomendable solicitar al equipo de CloudBrokers un incremento de recursos de memoria y CPU al respectivo proyecto en Openshift. A saber, los proyectos en Openshift son:

- QA: sentenciasjudiciales-preprod-axa-seg-mx
- Prod: sentenciasjudiciales-prod-axa-seg-mx

Dentro de estos proyectos, se encontrarán los pods que mantienen el servicio de la solución, si se busca mayor detalle sobre alguna incidencia, se recomienda tener amplia observabilidad sobre el ambiente afectado. Se recomienda que a partir de las incidencias en ambientes de Prod y QA, se procure replicar el error presentado en un ambiente de desarrollo donde se podrá trabajar la solución.

Los registros pueden mostrar cualquier incidencia en el código, y es posible que estas estén ligadas a servicios externos, tal como SecureGPT o servicios de Azure. Para cada servicio, es importante validar la operación, accesos y permisos con los equipos correspondientes:

- Servicios de Azure: Equipo de AXAGO (Temas relacionados con servicios en la nube como CosmosDB, AI Search, PrivateEndpoints, BlobStorage, etc)

Internal

- Servicios de Openshift: Equipo de Pipelines CICD (Temas relacionados con los contenedores, sus recursos y las reglas comunicación de redes)
- Servicios de SecureGPT: Consultar con el equipo internacional encargado de SecureGPT cualquier tema sobre el servicio, para más información sobre este servicio puede consultarlo en Welcome to our documentation! | Secure GPT Model Hub

Rollback

El proyecto consta de un conjunto de repositorios en AzureDevOps, bajo la organización AXA-MX-TECNOL-INFORMACION (https://dev.azure.com/AXA-MX-TECNOL-INFORMACION), el cual está compuesto de 5 repositorios de distintos servicios del sistema:

- D-JDK-291: Sentencias Judiciales (Frontend)
- SRV-PYT-783: Tratamiento de información documental
- SRV-PYT-784: Despersonalización de documentos
- SRV-PYT-785: Asistente conversacional Chatbot Sentencias Judiciales
- SRV-AZOCR-788: Motor de reconocimiento óptico de caracteres (OCR)

Para todos estos repositorios se seguirá el mismo proceso de Rollback correspondiente a cada servicio en su respectivo repositorio. Todo esto acompañado de los respectivos procesos de liberación de AXA, tal como generar una solicitud de liberación del servicio en el ambiente afectado.

Todos estos repositorios mantienen el mismo esquema de ramas de desarrollo:

- **Ramas principales:**
  - main: Rama estable para producción.
  - qa: Rama para pruebas antes de producción.

Internal

- dev: Rama de desarrollo activo.

- **Ramas para el desarrollo:**
  - feature/*: Ramas para nuevas funcionalidades.
  - hotfix/*: Ramas para correcciones urgentes en producción.

Asegúrate de seguir la convención de ramas y realizar pull requests para integrar cambios a develop o main.

Proceso: Portal AzureDevOps

Es posible revertir el estado anterior del código, y por lo tanto su nuevo despliegue, a través del portal de Azure DevOps. El método “Revertir” en Azure DevOps es el equivalente a un ‘git revert’ en la línea de comandos. Es el enfoque más seguro porque crea un nuevo commit que deshace los cambios de uno o más commits anteriores, manteniendo intacto el historial original del repositorio.

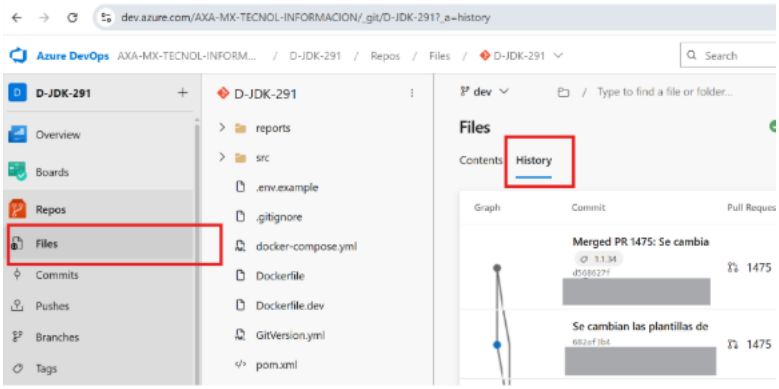
Pasos para un Rollback con la función “Revertir” en Azure DevOps

1. Navega al repositorio (https://dev.azure.com/AXA-MX-TECNOL-INFORMACION):
   a. Inicia sesión dentro de la organización.
   b. Ve al proyecto donde se encuentra el repositorio del servicio que se desea revertir.
   c. En el menú de la izquierda, selecciona Repos > Files.
2. Asegúrate de estar en la rama ‘main’ si lo que se desea es revertir la versión productiva. En la parte superior izquierda, asegúrate de que el selector de ramas muestre ‘main’, o la rama que se desee revertir.

Internal



3. Identifica los commits a revertir:
    a. Ve a la pestaña historial (dentro de Repos)



    b. Aquí verás la lista de commits de tu rama. Identifica el commit estable al que quieres volver. Es decir, necesitas encontrar el último commit "*bueno*" antes de que los problemas comenzaran.

    c. Localiza los commits que quieres deshacer (generalmente, serán los commits *después* de tu commit estable hasta el más reciente)

4. Inicia el proceso de Reversión:
    a. Para revertir un único commit: si lo que se desea es deshacer un singular commit específico:
        i. Entrar al detalle del commit que se desea revertir.

Internal



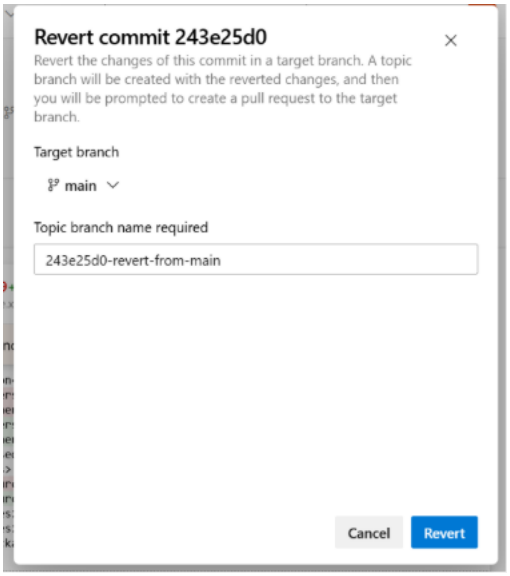
Dentro del detalle del commit, arriba de la pantalla aparecerán tres puntos con el detalle “More actions”. Este menú contendrá la opción “Revert”.

iii. Esta opción creará un nuevo commit que borrará los cambios aplicados en el commit a revertir.

5. Configura la Reversión:

   a. Se abrirá una ventana de dialogo, la cual solicitará la creación de una nueva rama en donde se aplicará primeramente la reversión del cambio. Es posible probar desde esta rama si el código efectivamente se ha revertido a un estado estable.

   b. Es importante darle un nombre significativo a la rama, como “rollback/main-to-commit-12345”, y proceder con el botón “Revert”



6. Confirma y revisa:

Internal

a. Una vez creada la nueva rama con el commit revertido, AzureDevOps te indicará que la reversión se ha completado y te dará un enlace a la nueva rama, donde se tendrá que abrir un Pull Request para hacer el merge de vuelta a 'main'.

b. Una vez aprobado el Pull Request de nuestra rama productiva, el pipeline de despliegue realizará las actividades de construcción de la nueva imagen con el código actualizado. Valida que todos los procesos del pipeline se completen correctamente.

Una vez validados los cambios y realizado el merge correspondiente de la rama con el código resuelto, la construcción de la imagen, así como el despliegue de la solución se realizará de manera automática.

Internal

SecureGPT Model Hub

Incidencias.

IMPORTANTE:

Si la API de Secure GPT sufre incidencias y/o caídas, la aplicación IA Sentencias Judiciales sufrirá de lo siguiente:

- **Incapacidad de cargar documentos.** Los documentos se cargan en la base de conocimientos usando los modelos de embeddings, así que, cuando Secure GPT esté caído, la obtención del modelo de embeddings arrojará error, por lo tanto, la aplicación será incapaz de cargar documentos.
- **Incapacidad de interactuar con el asistente.** El asistente conversacional es realmente un modelo largo de lenguaje consultado a través Secure GPT, si la API de este se encuentra disfuncional, entonces el asistente conversacional también lo estará.

¿Cómo proceder en caso de que se retire alguno o algunos de los modelos de Secure GPT Model HUB que usa la aplicación?

Para entender mejor cómo proceder cuando los modelos de Secure GPT Model HUB que usa la aplicación responderemos a las siguientes preguntas:

- ¿Cuáles son los modelos del Secure GPT Model Hub que actualmente estamos utilizando en el asistente?

Internal

- En caso de que algún modelo del Secure GPT Model Hub que estamos empleando sea retirado, ¿cuáles serían las implicaciones para el funcionamiento del asistente?
- ¿Es necesario realizar alguna acción en particular ante el retiro de un modelo de Secure GPT Model Hub?
- ¿Cuáles son los modelos del Secure GPT Model Hub que actualmente estamos utilizando en el asistente?

¿Cuáles son los modelos del Secure GPT Model Hub que actualmente estamos utilizando en el asistente?

Al momento de la escritura de la presente documentación (agosto, 2025), la solución IA Sentencias Judiciales utiliza en sus servicios SRV-PYT-783 y SRV-PYT-785, los siguientes modelos:

Modelo de Embeddings:

- SECURE GPT ENDPOINT BASE: https://api.se.axa-go.applications.services.axa-tech.intraxa/ago-m365-securegpt-hub-v1-vrs
- SECURE GPT VERSIÓN DE API: 2024-10-21
- ID DEL MODELO EN SECURE GPT: text-embedding-3-large-1
- NOMBRE DEL MODELO: text-embedding-3-large
- DIMENSION: 1024

Modelo Grande de Lenguaje:

- SECURE GPT ENDPOINT BASE: https://api.se.axa-go.applications.services.axa-tech.intraxa/ago-m365-securegpt-hub-v1-vrs
- SECURE GPT VERSIÓN DE API: 2024-10-21
- ID DEL MODELO EN SECURE GPT: gpt-4o-mini-2024-07-18
- NOMBRE DEL MODELO: gpt-4o-mini

Internal

En caso de que algún modelo del Secure GPT Model Hub que estamos empleando sea retirado, ¿cuáles serían las implicaciones para el funcionamiento del asistente?

Cuando alguno de los modelos que utiliza la aplicación es retirado de Secure GPT Model Hub, entonces la aplicación NO estará disponible y arrojará error al momento de consumir dichos modelos.

¿Es necesario realizar alguna acción en particular ante el retiro de un modelo de Secure GPT Model Hub?

La respuesta a esta pregunta es un contundente sí. A continuación, se explica el proceso que debe seguirse para "migrar/cambiar" alguno de los modelos que la aplicación usa.

Embeddings

Para cambiar/sustituir/reemplazar el modelo de embeddings se procede con los pasos siguientes:

- Visita Secure GPT Model Hub: Welcome to our documentation! | Secure GPT Model Hub
- Revisa la sección "Offerings > Models": Models | Secure GPT Model Hub
- Identifica el modelo de interés y recolecta la siguiente información:
    - SECURE GPT ENDPOINT BASE.
    - SECURE GPT VERSIÓN DE API.
    - ID DEL MODELO EN SECURE GPT.
    - NOMBRE DEL MODELO.
    - DIMENSION.

Internal

IMPORTANTE:

La dimensión del modelo de embeddings con la que nació el proyecto tiene el valor 1024. Esto implica que cualquier modelo seleccionado para sustituir debe tener, forzosamente, dimensión 1024, si esto no pasa, la base de conocimientos se romperá y el resultado implicará una recarga completa de todos los documentos.

Modelo Largo de Lenguaje (LLM)

Para cambiar/sustituir/reemplazar el modelo de embeddings se procede con los pasos siguientes:

- Visita Secure GPT Model Hub: Welcome to our documentation! | Secure GPT Model Hub
- Revisa la sección "Offerings > Models": Models | Secure GPT Model Hub
- Identifica el modelo de interés y recolecta la siguiente información:
    - SECURE GPT ENDPOINT BASE.
    - SECURE GPT VERSIÓN DE API.
    - ID DEL MODELO EN SECURE GPT.
    - NOMBRE DEL MODELO.

IMPORTANTE:

El modelo que sustituya al LLM debe ser capaz de devolver respuestas en streamming.

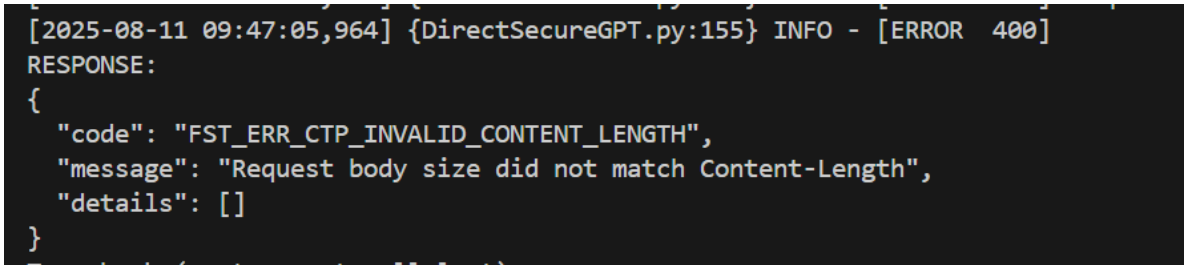
Incidencia en el servicio con fecha 11 agosto 2025

Durante el desarrollo del aplicativo se presentó la incidencia de SecureGPT siguiente:

- ENDPOINT SECURE GPT QA/PREPROD: https://api-pp.se.axa-go.applications.services.axa-tech.intraxa/ago-m365-securegpt-hub-v1-vrs

Internal

- FECHA DE INCIDENCIA: 8 agosto 2025.
- Descripción de Incidencia: El endpoint devuelve un código 400 en el que se enuncia : “Request body size did not match Content-Length”.
- IMAGEN DE INCIDENCIA EN FORMA DE LOG:



Internal

Flujos ETL

En esta sección se detallan lo desarrollos del servicio SRV-PYT-783 concentrados en procesos Extract Transform and Load que cumplen con los siguientes dos objetivos:

- Crear y actualizar la tabla de sentencias cargadas en la aplicación, que a su vez alimenta el tablero de PowerBI. (ETL Tablero).
- Eliminar los datos asociados a sentencias cuya antigüedad es mayor a diez años. (ETL Limpieza).

ETL Tablero

Inicialmente en el proyecto PDP-767 IA Sentencias Judiciales se planteó una arquitectura en la cual el tablero que permite visualizar insights de los documentos cargados a la base de conocimiento se obtenía a través de la capa Gold del datalake. Sin embargo, durante el proceso se optó por otra alternativa de consumo de datos para la generación dicho tablero **BI Indicadores de Éxito Sentencias en Curso** con código asignado **BI-040-253**, en la cual se crea una tabla de tipo csv que será almacenada en el mismo contenedor donde actualmente se cargan las sentencias despersonalizadas, es decir en *censored-documents,* pero en la carpeta *table-metadata.*

Dicha tabla cumple con el formato mostrado abajo y ya toma en cuenta las transformaciones necesarias para el consumo directo de PowerBI, únicamente se muestra un registro para dar a conocer la estructura:

	FILE_ID	CIRCUIT	LOCALITY	COURT_NUMBER_STR	COURT_MATTERS	COURT_MATTERS_STR	AUTORITY	YEAR	MONTH	DAY	DATE	LOST_CASE
0	8ad002fd7092595b4164b5e31e5c824c689c6bf4f193a0...	1	CIUDAD DE MEXICO	JUZGADO TRIGESIMO CUARTO	CIVIL	CIVIL	JUZGADO TRIGESIMO CUARTO EN MATERIA CIVIL	2021	6	3	2021-06-03	True

La implementación de esta solución se acordó después del comienzo de la carga histórica, por ello es necesario crear una tabla inicial, la cual es la versión de consumo para PowerBI a partir de los metadatos ya almacenados en CosmosDB.

En este sentido, el primer paso será ejecutar el endpoint **document/create-table**, el cual se puede ejecutar, según se necesite, en cualquiera de los ambientes desde Swagger:

- Swagger SRV-PYT-783 DEV: http://localhost:8783/docs

Internal

- Swagger SRV-PYT-783 QA: https://srv-pyt-783-sentenciasjudiciales-preprod-axa-seg-mx.axa-seg-mx-preprod-lpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

- Swagger SRV-PYT-783 PROD: https://srv-pyt-783-sentenciasjudiciales-prod-axa-seg-mx.axa-seg-mx-prod-hpl-int.gold.eastus2.azure.openpaas.axa-cloud.com/docs

Una vez ejecutado el endpoint, se creará el documento axa-ai-sentencias-judiciales/table-metadata/sentence_table.csv dentro del contenedor censored-documents con la estructura de la imagen descrita arriba. Para más detalle sobre los campos se puede revisar la sección **Datos que alimentan el tablero** en **BI-040-253 Indicadores de éxito para sentencias en curso.**

Además, cada que un documento sea cargado en la base de conocimiento, un registro con los metadatos para alimentar PowerBI será añadido a dicho csv. El proceso es análogo cuando un documento se elimina de la base de conocimiento.

IMPORTANTE

- Si la tabla inicial (cuando ya haya metadatos en CosmosDB) no ha sido generada, al cargar un documento en la base de conocimiento este será el primero registro en la tabla lo cual hará que la información antes de esa carga no se vea reflejada en la tabla.

- Si no hay metadatos en CosmosDB la ejecución del endpoint de creación de la tabla generará una tabla vacía.

Internal

ETL Limpieza

El presente flujo de datos ha sido nombrado ETL Limpieza, pues como parte del servicio SRV-PYT-783 del proyecto PDP 767 IA SENTENCIAS JUDICIALES es necesario limpiar contenidos de la base de conocimiento basándose en la antigüedad de la sentencia.

El objetivo de este flujo es ejecutar el flujo de limpieza de información con antigüedad mayor a diez años.

Para el cálculo de la antigüedad se siguió la siguiente lógica:

- El cálculo de la antigüedad será realizado utilizando la fecha de resolución de la sentencia y se comparará contra la fecha actúa; esta última es obtenida al momento de la ejecución del flujo.
- Cuando la fecha de resolución no pudo ser obtenida, entonces se comparará la fecha actual contra la fecha en la que se cargó el documento en la plataforma.
- Para ambos casos, si la diferencia entre las fechas es mayor a 10 años, entonces ese documento debe ser eliminado de todos los servicios de almacenamiento que forman parte del proyecto, incluyendo el archivo CSV del blob storage que alimenta el tablero de powerbi.

Las funciones y scripts desarrollados para el flujo se encuentran en la siguiente carpeta:

- app/api/core/cleanup.

Los archivos py definidos son los siguientes:

- app/api/core/cleanup/cleanup_data_flow.py
- app/api/core/cleanup/execute_cleaning_process.py

Funcionalidades por archivo:

- app/api/core/cleanup/cleanup_data_flow.py
    - Se definen las funciones de lectura, evaluación y borrado de información.
- app/api/core/cleanup/execute_cleaning_process.py

Internal

- Se define un script que ejecuta el flujo de limpieza a través de las funciones definidas en: app/api/core/cleanup/cleanup_data_flow.py

Además de esto, se añade a la aplicación FastAPI el endpoint siguiente:

- ENDPOINT: /clean-up-data/execute
- TIPO: DELETE

Dicho endpoint ejecuta el flujo de limpieza a través de las funciones definidas en: app/api/core/cleanup/cleanup_data_flow.py

Como parte del desarrollo de dicho flujo de actualización, al final de cada ejecución de dicho flujo, se generan dos archivos CSV que sirven como base para la revisión del proceso de ejecución. Estos archivos se generan siguiendo la siguiente lógica:

- Se genera un UUID y se etiqueta como “ID DE EJECUCIÓN”.
- Se ejecuta el proceso de actualización. Durante este proceso, se recopila la información siguiente:
  - Para cada uno de los documentos de emtadatos cargados en la colección “document_metadata” de CosmosDB. Se obtienen los campos:
    - file_id
    - hash
    - upload_timestamp
    - sentence_resolution_data.ruling_date
  - Se registran los posibles errores en el campo sentence_resolution_data.ruling_date.
  - Se evalúan dichos errores:
    - Si los errores hacen que el campo sentence_resolution_data.ruling_date sea ambiguo entonces se define como fecha de referencia el campo upload_timestamp.
    - En caso contrario se usa como referencia sentence_resolution_data.ruling_date

Internal



    - Se evalúa y registra la antigüedad del documento comparando la fecha de referencia contra la fecha actual.

- Con los documentos identificados como “viejitos” por su antigüedad, se ejecuta el proceso de eliminación en las bases de datos.
- Con la información recopilada, se generan dos tablas, cada una de ellas será guardada en archivo un csv en blbo storage:
    - Archivo 1:
        - Nombre de archivo: Evaluated_docs.csv
        - Ruta en blob storage: censored-documents/axa-ai-sentencias-judiciales/actualization_reports/{execution_year}/{execution_month}/{execution_id}/Evaluated_docs.csv
        - Descripción: En esta tabla se listan todos los documentos que viven en CosmosDB así como información asociada a los mismos y relacionada con la fecha de resolución y la fecha de carga.
        - Campos de la tabla:
            - file_id: El id asociado al documento.
            - original_ruling_date: La fecha original de resolución de sentencia.
            - evaluation_date: La fecha utilizada para evaluar la antigüedad.
            - error_on_year: Booleano que indica si el error de formateo fue en el año.
            - error_on_month Booleano que indica si el error de formateo fue en el mes.
            - error_on_day: Booleano que indica si el error de formateo fue en el día.
            - drop: Booleano que indica si el documento.

Internal

- Archivo 2:
    - Nombre de archivo: Dropped_docs.csv
    - Ruta en blob storage: censored-documents/axa-ai-sentencias-judiciales/actualization_reports/{execution_year}/{execution_month}/{execution_id}/Dropped_docs.csv
    - Descripción: Este CSV contiene información de los documentos que debieron haber sido eliminados por su antigüedad. Cuando el flujo NO identifica documentos para eliminar, entonces la información se resume en los siguientes capos y valores:
        - execution_id: El ID de la ejecución.
        - execution_timestamp: La estampilla de tiempo de la ejecución.
        - MESSAGE: Literalmente se enuncia “No se encontraron documentos para eliminar”
    - Campos de la tabla:
        - file_id: El id asociado al documento.
        - original_ruling_date: La fecha original de resolución de sentencia.
        - evaluation_date: La fecha utilizada para evaluar la antigüedad.
        - error_on_year: Booleano que indica si el error de formateo fue en el año.
        - error_on_month Booleano que indica si el error de formateo fue en el mes.
        - error_on_day: Booleano que indica si el error de formateo fue en el día.
        - drop: Booleano que indica si el documento.

Internal

Anexo

Sentencia antes y después del procesamiento

Muestra de una sentencia dummy antes y después de la despersonalización

PODER JUDICIAL DE LA FEDERACIÓN

AMPARO DIRECTO: 123/2024

MATERIA: INCUMPLIMIENTO DE CONTRATO DE SEGURO

QUEJOSO: JUAN PÉREZ MARTÍNEZ

DEMANDADO: AXA SEGUROS, SOCIEDAD ANÓNIMA DE CAPITAL VARIABLE

MAGISTRADO PONENTE: LIC. JOSÉ LUIS RAMÍREZ HERNÁNDEZ
SECRETARIO: LIC. MARÍA FERNANDA GÓMEZ

Ciudad de México, México, acuerdo al Primer Tribunal Colegiado en Materia Civil del Primer Circuito, correspondiente al día **quince de marzo de dos mil veinticinco**

VISTOS para resolver el juicio promovido por el C. Juan Pérez Martínez en contra de AXA SEGUROS, S.A. DE C.V., por incumplimiento de contrato de seguro de gastos médicos mayores, y considerando los elementos de prueba aportados por las partes, se emite la presente resolución.

RESULTANDO:

Internal

PODER JUDICIAL DE LA FEDERACIÓN

AMPARO DIRECTO: 123/2024

MATERIA: INCUMPLIMIENTO DE CONTRATO DE SEGURO

QUEJOSO: [redacted]

DEMANDADO: [redacted] SEGUROS, SOCIEDAD ANÓNIMA DE CAPITAL VARIABLE

MAGISTRADO PONENTE: [redacted]

SECRETARIO: [redacted]

[redacted] de México, México, acuerdo al Primer Tribunal Colegiado en Materia Civil del Primer Circuito, correspondiente al día **quince de marzo de dos mil veinticinco**

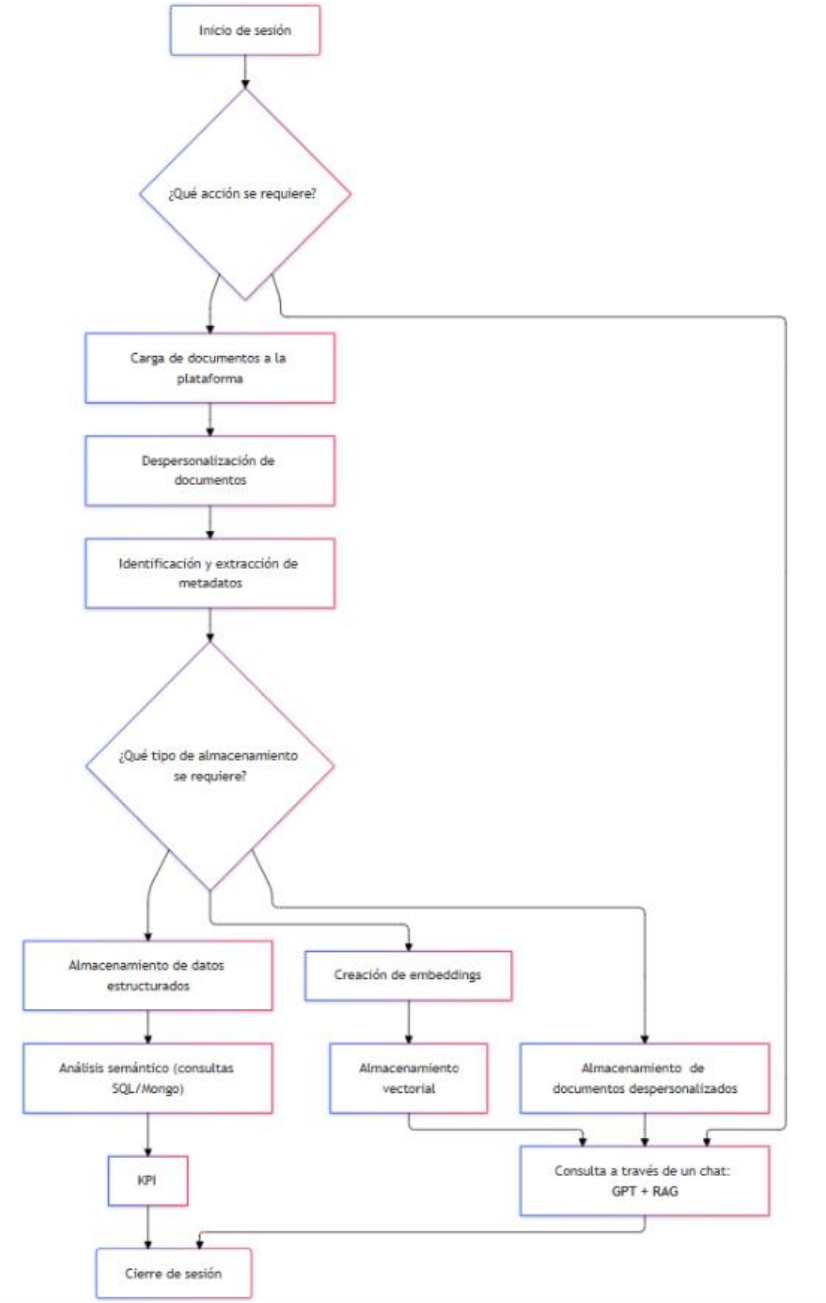
VISTOS para resolver el juicio promovido por el C. [redacted] en contra de [redacted] S.A. DE C.V., por incumplimiento de contrato de seguro de gastos médicos mayores, y considerando los elementos de prueba aportados por las partes, se emite la presente resolución.

RESULTANDO:

Internal

Diagrama de secuencia

El flujo general de la herramienta y la interacción de estos módulos puede observarse en el siguiente diagrama:



Internal

Prompts de templates utilizados

A continuación, se muestran unos ejemplos de Prompts que se encuentran integrados dentro del asistente en el apartado de “Mis conversaciones guardadas”

Prompt General

Hola, estoy involucrado en una inconformidad de parte de un cliente de AXA Seguros México. Yo soy el encargado de brindar la primera atención a dicho reclamo y atenderlo.

El reclamo cuenta con las siguientes características:

- <Característica 1>
- <Característica 2>
- <Característica 3> ... (Presione Shift + Enter para agregar más características)

Quisiera conocer si se han presentado casos similares en el pasado y como podría brindar una respuesta a esta inconformidad basándome en el contrato como cláusulas, excepciones, etc, así como conocer la tasa de éxito.

Dame un resumen con esta información y en tu respuesta actúa como una persona con conocimiento en atención al cliente.

Prompt Legal

Hola, soy el abogado encargado de dar contestación a una demanda en contra de AXA Seguros, en el cual yo me encargo de establecer un camino legal a tomar para que AXA salga favorecida en la sentencia emitida.

El caso que tengo actualmente tiene las siguientes características:

- <Característica 1>
- <Característica 2>
- <Característica 3> ... (Presione Shift + Enter para agregar más características)

Necesito que me ayudas a encontrar casos similares y a partir de ellos busco:

- <Respuesta esperada 1>
- <Respuesta esperada 2>
- <Respuesta esperada 3> ... (Presione Shift + Enter para agregar más respuestas esperadas)

Dame un resumen con esta información y en tu respuesta actúa como una persona con conocimiento en leyes.

Prompt Médico

Hola, actualmente estoy involucrado en una inconformidad por parte de un cliente de AXA Seguros México. En esta disputa yo soy el dictaminador que revisa las controversias. Actualmente estoy tratando de conocer los alcances en decisiones judiciales basados en sentencias emitidas previamente, y que tienen características similares a la inconformidad que atiendo actualmente. La inconformidad actual tiene las siguientes características:

- <Característica 1>
- <Característica 2>
- <Característica 3> ... (Presione Shift + Enter para agregar más características)

Con las sentencias similares previas podrías:

- <Respuesta esperada 1>
- <Respuesta esperada 2>
- <Respuesta esperada 3> ... (Presione Shift + Enter para agregar más respuestas esperadas)

Internal

Troubleshooting

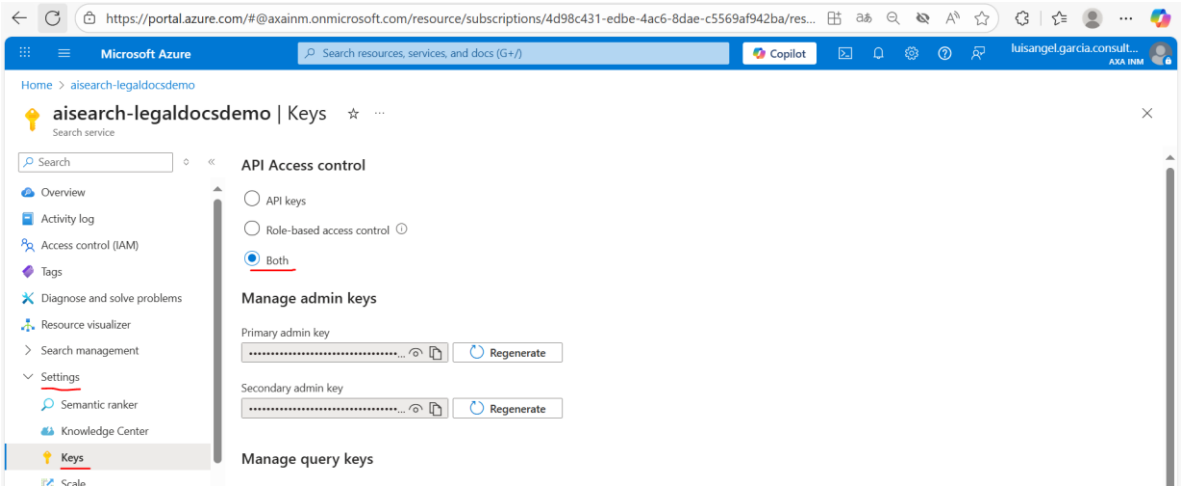
En esta sección abordaremos los errores más comunes que se dan en el aplicativo PDP 767 IA Sentencias Judiciales.

403 Forbbiden – Azure AI Search

Cuando el servicio SRV-PYT-783 o el servicio SRV-PYT-785 marcan un error que enuncia: 403 Forbbiden y que es derivado de la conexión hacia AI Search, entonces, deberemos contactar al equipo AXA GO y solicitar la revisión de la configuración de AI Search siguiente:

- Tenant: AXA 365
- Grupo de recursos: z-nsm-iasentenciasjud-pr01-ue2-01
- Nombre de recurso: znsmiasejupreu2ass01
    - Dentro del recurso AI Search ir a:
        - Setting > Keys.
    - Validar con equipo AXA Go que la configuración API Access Control esté establecida en la opción “Both”

Ejemplo de correcta configuración:

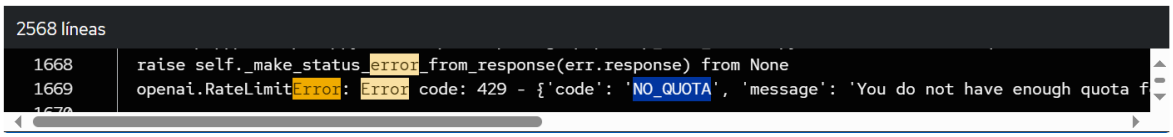


Internal

429 NO_QUOTA – Secure GPT

Cuando el servicio SRV-PYT-783 o el servicio SRV-PYT-785 registran un error que enuncia lo siguiente:

openai.RateLimitError: Error code: 429 - {'code': 'NO_QUOTA', 'message': 'You do not have enough quota for this request', 'details': [{'location': 'Quota[gpt-4o-mini-2024-07-18][tokens]', 'message': 'Insufficient quota for this type'}]}



Entonces se trata de un error por causas del límite de consumo de tokens de la API de SecureGPT.

Para proceder con el aumento de quotas de la API, deben seguirse los lineamientos enlistados en el siguiente enlace:

You can raise your quotas using this request Item - Employee Service Center, it should be filled by your local contacts (☎ Local contact - Secure GPT - Confluence by AXA GO)

Internet Connection Failed

Si durante el proceso de carga de un documento o la ejecución de un mensaje hacia el asistente ocurre que nuestro equipo pierde la conexión a internet, entonces:

- Para el caso del documento que se está cargando, el proceso de carga NO se cancela, más bien, su ejecución continúa en el backend. Derivado de los altos tiempos de procesamiento del proceso de carga, entonces esperemos 30min, y busquemos nuestro documento en la sección de consulta de información si no está presente, entonces procedemos a recargarlo.
- Para el caso del mensaje enviado en la conversación:
    - Si la conversación no fue guardada previamente, dicha conversación se perderá.

Internal

- Si la conversación SÍ fue guardada previamente, esperemos un lapso de 5min antes de recuperar la conversación desde la lista de conversaciones guardadas, el mensaje de la IA deberá aparecer en ella.

Internal